



République Tunisienne
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
Université de Tunis El Manar
École Nationale d'Ingénieurs de Tunis



Conception d'un pipeline MLOps pour la maintenance prédictive d'un système mécanique

Réalisé par :

Bourbia Othmen

Boubakri Fares

Bourbia Said

Encadré par :

Mme Wafa mefteh

Année universitaire :2025/2026

Remerciements

Nous tenons à exprimer notre profonde gratitude à notre encadrante, **Madame Wafa mefteh**, pour son accompagnement continu, ses conseils précieux et sa disponibilité tout au long de la réalisation de ce projet. Son expertise scientifique et sa rigueur méthodologique ont largement contribué à la réussite de ce travail.

Nous adressons également nos sincères remerciements à l'ensemble du corps enseignant et administratif de l'École Nationale d'Ingénieurs de Tunis pour la qualité de la formation dispensée et le soutien offert durant notre parcours académique.

Enfin, nous remercions toutes les personnes qui ont contribué, de près ou de loin, à la réussite de ce projet.

Résumé

Ce rapport présente sur le développement d'un pipeline MLOps pour la prédiction de la durée de vie utile restante (RUL) des moteurs turboréacteurs. La maintenance prédictive est devenue un enjeu stratégique majeur dans le domaine aéronautique, permettant de réduire les arrêts non planifiés et d'assurer la sécurité des vols par l'analyse des données de capteurs en temps réel. Notre approche exploite Machine Learning et le Deep Learning pour développer des modèles de régression capables d'estimer précisément la RUL des turbofans. Nous avons implémenté une architecture MLOps complète intégrant un système de prétraitement de données, d'entraînement et de validation des modèles, de déploiement containerisé, et de monitoring continu avec détection de dérive. L'objectif principal est de concevoir et d'évaluer sept architectures de Deep Learning différentes sur le dataset C-MAPSS (Commercial Modular Aero-Propulsion System Simulation) fourni par la NASA.

Mots clés : Maintenance prédictive, RUL, Deep Learning, MLOps, C-MAPSS, Pipeline de données, Déploiement, Turboréacteur.

Table des matières

Liste des figures

Liste des tableaux

Liste des abréviations

Introduction générale	1
1 Contexte général et cadre théorique	2
1 Introduction	3
2 Système mécanique	3
3 Stratégies de maintenance	4
3.1 Les types de maintenance	4
4 Maintenance prédictive basée sur les données	5
5 APPRENTISSAGE AUTOMATIQUE (ML)	7
5.1 DEFINITION	7
5.2 TYPES APPRENTISSAGE AUTOMATIQUE	7
5.2.1 APPRENTISSAGE SUPERVISE	8
5.2.2 APPRENTISSAGE NON SUPERVISE	11
6 Deep Learning pour la maintenance prédictive	13

TABLE DES MATIÈRES

6.1	Réseaux récurrents et mécanismes d'attention	13
6.1.1	Architectures hybrides	14
7	Conclusion	15
2	Analyse et conception	17
1	Introduction	18
2	Présentation du système étudié	18
2.1	Description du système étudié	18
2.1.1	Architecture et fonctionnement du turboréacteur	18
2.1.2	Surveillance par capteurs et prédiction des défaillances	19
2.2	DESCRIPTION DE LA BASE DONNEES	20
2.2.1	Sous-ensembles du jeu de données C-MAPSS	20
3	Conception globale du système	21
3.1	Architecture générale du pipeline MLOps	21
3.2	Flux de données	22
4	Modèles de Deep learning pour la maintenance prédictive	28
4.1	Étude des modèles de Deep learning appliqués à la maintenance prédictive	28
4.2	Critères de sélection et évaluation des performances des modèles	33
4.2.1	Métriques utilisées	33
4.2.2	Évaluation des performances des modèles	36
5	Conclusion	36

TABLE DES MATIÈRES

3	Réalisation et tests	37
1	Introduction	39
2	Environnement technique et outils	39
2.1	Langage	39
2.1.1	Python	39
2.2	Bibliothèques	40
2.2.1	PyTorch	40
2.2.2	Streamlit	40
2.2.3	scikit-learn	41
2.2.4	Numpy	41
2.3	Environnement de développement	42
2.3.1	Visual Studio Code (VS Code)	42
2.3.2	Git	42
2.3.3	Hugging Face Spaces	43
2.3.4	Docker Desktop	43
3	Préparation et prétraitement des données	44
3.1	Prétraitement des données pour la prédiction RUL	44
3.2	Préparation des données pour les modules annexes (anomalies, SHAP))	46
4	Développement des services applicatifs	46
4.1	API REST avec FastAPI	46
4.1.1	Architecture générale et endpoints exposés	46
4.1.2	Fonctionnement technique	47
4.2	Dashboard interactif avec Streamlit	47
5	Conteneurisation avec Docker	48

TABLE DES MATIÈRES

5.1	définition et objectifs	48
5.1.1	Implémentation dans le projet	48
5.1.2	Architecture mono-conteneur (API + dashboard + Nginx)	48
5.1.3	Élaboration du Dockerfile et construction de l'image . . .	49
5.1.4	Orchestration interne : Nginx et Supervisor	50
5.1.5	Construction et test local de l'image	51
6	Déploiement sur Hugging Face Spaces	52
6.1	Création du Space Docker et configuration du dépôt	52
6.2	Versionnement et push du code via Git	54
6.2.1	Initialisation du dépôt local et liaison avec le Space	54
6.2.2	Push du code et déclenchement automatique du build . . .	54
6.2.3	Suivi du build et analyse des logs	55
7	Validation des modèles et analyse des résultats	55
7.1	Résultats comparatifs	55
7.1.1	Tableau des performances	56
7.2	Analyse approfondie des résultats par modèle	56
7.2.1	Modèle LSTM	56
7.2.2	Modèle CNN-LSTM	57
7.2.3	Modèle Transformer	57
7.2.4	Analyse des erreurs résiduelles	57
8	Conclusion	58
Conclusion Générale		58
Bibliographie		60

Liste des figures

1.1	Exemple d'un système mécanique	3
1.2	Différents scénarios de maintenance [2]	4
1.3	Les types d'apprentissage automatique	8
1.4	Illustration de l'apprentissage supervisé [3]	9
1.5	Illustration de l'apprentissage non supervisé [3]	12
1.6	Architecture hybride CNN 1D – LSTM pour la classification des signaux — du CNN extracteur de caractéristiques au LSTM modélisateur de dépendances temporelles	15
2.1	Vue d'un turboréacteur double flux	19
2.2	Architecture générique d'un pipeline MLOps : de l'ingestion des données au déploiement du modèle avec boucles de rétroaction	22
2.3	Flux de données du système de maintenance prédictive (C-MAPSS)	22
2.4	Structure et organisation du jeu de données C-MAPSS (NASA)	23
2.5	Diagramme des vérifications de validation et contrôle de qualité	24
2.6	Diagramme du prétraitement et extraction de caractéristiques	25
2.7	Diagramme de l'entraînement et validation du modèle	26
2.8	Diagramme du déploiement et inférence	27
2.9	Architecture du modèle LSTM pour la prédiction de la durée de vie restante (RUL) à partir des données C-MAPSS	29

2.10	Architecture du modèle CNN-1D pour l'estimation de la durée de vie restante (RUL) à partir des données C-MAPSS	30
2.11	Architecture du modèle BiLSTM pour l'estimation de la durée de vie restante (RUL) à partir des données C-MAPSS	31
2.12	Architecture du modèle Transformer pour l'estimation de la durée de vie restante (RUL) à partir des données C-MAPSS	32
2.13	Architecture hybride combinant des couches convolutives (CNN), récurrentes (LSTM) et un module Transformer (mécanisme d'attention) pour la prédiction de la durée de vie restante.	33
3.1	logo python	39
3.2	logo PyTorch	40
3.3	logo StreamLit	41
3.4	logo scikit-learn	41
3.5	logo Numpy	42
3.6	logo VSCode	42
3.7	logo Git	43
3.8	Hungging face logo	43
3.9	logo Docker Desktop	43
3.10	Les étapes de prétraitement des données	44
3.11	Contenu du Dockerfile utilisé dans le projet	49
3.12	Contenu du Dockerfile utilisé dans le projet	50
3.13	Orchestration interne	50
3.14	Commande de lancement du conteneur en local	51
3.15	Réponse de l'API <code>/api/health</code> en local	51
3.16	Dashboard Streamlit accessible en local	52

3.17	Création du Space Docke	53
3.18	Initialiser un dépôt Git local	54
3.19	Ajouter tous les fichiers	54
3.20	Ajouter le dépôt distant du Space Hugging Face	54
3.21	Pousser le code vers la branche principale du Space	54
3.22	logs	55

Liste des tableaux

2.1	Composition et caractéristiques des sous-ensembles C-MAPSS	20
3.1	Endpoints de l'API REST	47
3.2	Performances des modèles sur le test externe (hors drift et recommandations)	56

Liste des abréviations

LSTM : Long Short-Term Memory – Mémoire à court et long terme

CNN : Convolutional Neural Network – Réseau de neurones convolutif

RUL : Remaining Useful Life – Durée de vie utile restante

BiLSTM : Bidirectional LSTM – LSTM bidirectionnel

MLOps : Machine Learning Operations – Opérations d'apprentissage automatique

API : Application Programing Interface (Interface de programmation d'application)

ML : Machine learning – apprentissage automatique

Mp : maintenance prédictive

SHAP : SHapley Additive exPlanations – Explicabilité des modèles

RMSE : Root Mean Square Error – Racine de l'erreur quadratique moyenne

MAE : Mean Absolute Error – Erreur absolue moyenne

S-score : Scoring Function – Fonction de notation asymétrique pour RUL

SSH : Secure Shell – Protocole sécurisé d'accès distant

Introduction générale

La sécurité et la disponibilité des équipements constituent une priorité absolue dans l'industrie, en particulier dans le secteur aéronautique où la moindre défaillance peut avoir des conséquences critiques. En estimant la durée de vie utile restante (RUL) d'un composant, elle permet de réduire les temps d'arrêt imprévus, de minimiser les coûts et d'améliorer la sécurité tout en prolongeant la durée d'exploitation des machines.

Dans ce contexte, les turboréacteurs à double flux sont soumis à des dégradations progressives difficiles à modéliser par des lois physiques simples. Les méthodes d'apprentissage profond, telles que les CNN, LSTM ou Transformers, offrent une alternative efficace pour traiter la complexité de ces séries temporelles multivariées.

Ce projet de fin d'année vise à concevoir un pipeline MLOps dédié à la maintenance prédictive, en s'appuyant sur le jeu de données de référence C-MAPSS de la NASA. Ce pipeline couvre de l'ingestion des données jusqu'au déploiement d'un modèle d'estimation de la durée de vie utile restante (RUL). Afin d'identifier la solution offrant le meilleur équilibre entre précision et efficacité, une étude comparative rigoureuse a été menée sur plusieurs architectures d'apprentissage profond (LSTM, CNN-1D, BiLSTM, Transformer et modèles hybrides).

Le rapport présente d'abord un état de l'art, puis l'analyse et la conception du pipeline, et enfin la réalisation pratique avec les résultats expérimentaux et le déploiement du modèle.

Chapitre 1

Contexte général et cadre théorique

Sommaire

1	Introduction	3
2	Système mécanique	3
3	Stratégies de maintenance	4
3.1	Les types de maintenance	4
4	Maintenance prédictive basée sur les données	5
5	APPRENTISSAGE AUTOMATIQUE (ML)	7
5.1	DEFINITION	7
5.2	TYPES APPRENTISSAGE AUTOMATIQUE	7
6	Deep Learning pour la maintenance prédictive	13
6.1	Réseaux récurrents et mécanismes d'attention	13
7	Conclusion	15

1 Introduction

De nombreux secteurs industriels sont aujourd’hui transformés par l’intelligence artificielle, et la maintenance des systèmes mécaniques ne fait pas exception. Ce chapitre pose les bases théoriques de la maintenance prédictive en explorant les stratégies de maintenance, les fondamentaux du Machine Learning et les architectures de Deep Learning adaptées aux séries temporelles. L’objectif est de montrer comment l’apprentissage automatique fait évoluer une maintenance réactive ou périodique vers une prédiction intelligente des défaillances.

2 Système mécanique

Un système mécanique industriel correspond à un ensemble de composants interconnectés, tels que moteurs, engrenages et roulements, permettant d’assurer une fonction spécifique dans le processus de production [1]. Son fonctionnement repose sur la transformation et la transmission de l’énergie en mouvement utile afin de réaliser différentes opérations industrielles. Ces systèmes sont souvent soumis à des conditions de fonctionnement variables, ce qui peut entraîner une dégradation progressive de leurs performances. Afin de surveiller leur comportement, des capteurs sont intégrés pour mesurer des paramètres physiques tels que la température, les vibrations ou la pression [1]. Les données collectées permettent d’analyser l’état de fonctionnement des équipements et de détecter d’éventuelles anomalies. Ainsi, les capteurs jouent un rôle essentiel dans l’amélioration de la fiabilité et dans la mise en œuvre de stratégies de maintenance prédictive.

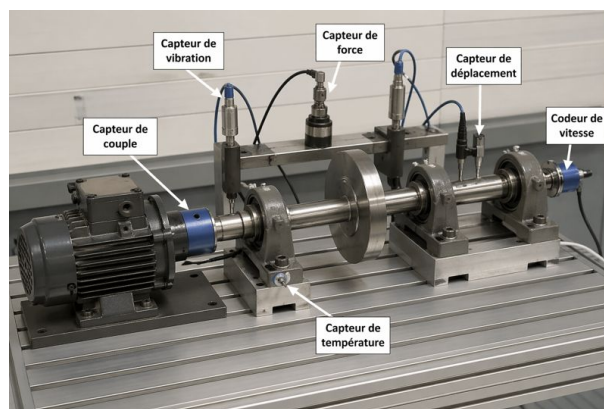


FIGURE 1.1 – Exemple d’un système mécanique

3 Stratégies de maintenance

Dans ce contexte, assurer la disponibilité et la fiabilité des machines devient un enjeu stratégique majeur. Les arrêts non planifiés peuvent engendrer des pertes financières importantes, des retards de livraison et une dégradation de la qualité des produits. Ainsi, la gestion efficace de la maintenance constitue un levier essentiel pour garantir la stabilité et la compétitivité des systèmes industriels modernes [2].

3.1 Les types de maintenance

La classification des activités de maintenance repose principalement sur le moment d'intervention par rapport à la survenue d'une défaillance [2]. Chaque type présente des caractéristiques, avantages et limites spécifiques qui influencent directement la performance du système de production et la qualité des produits. Nous distinguons trois catégories de stratégies de maintenance : la maintenance préventive, la maintenance corrective ou réactive et la maintenance prédictive.

La Figure 1.2 résume les scénarios de maintenance mentionnés.

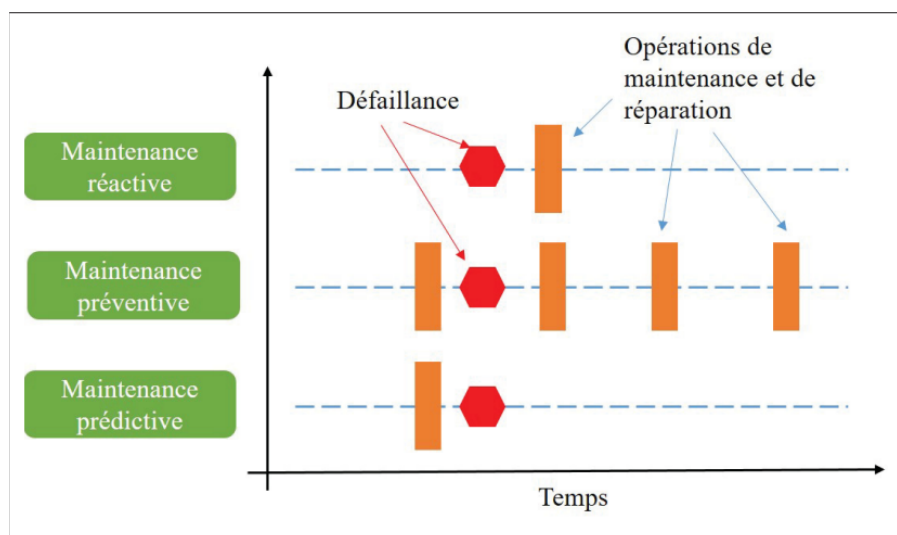


FIGURE 1.2 – Différents scénarios de maintenance [2]

Le scénario de **maintenance réactive** correspond à une approche dans laquelle les interventions sont réalisées uniquement après l'apparition d'une défaillance. Cette stratégie est considérée comme la plus coûteuse sur le plan de la perte de production des trois scénarios

de la Figure 1.2 , en raison des arrêts imprévus qu'elle engendre et de leur impact direct sur la disponibilité des équipements [2].

Le scénario de **maintenance préventive** constitue une amélioration par rapport à l'approche réactive, puisqu'elle repose sur la planification périodique des opérations de maintenance d'après la Figure 1.2 . Cette catégorie permet de réduire la probabilité de défaillance et d'améliorer la fiabilité des machines. Toutefois, dans les environnements de production flexibles, caractérisés par des changements fréquents de configuration et des cadences variables, l'application stricte de cette stratégie peut devenir contraignante et parfois inefficace [2].

D'après la Figure 1.2, Le scénario de **maintenance prédictive** c'est lorsque les opérations de maintenance ne sont plus effectuées périodiquement, mais déclenchées en fonction d'indicateurs reflétant l'état réel de l'équipement. Cette stratégie consiste à analyser les données de fonctionnement du système collectées à partir des capteurs dans le but de prédire la survenue de l'événement de défaillance [2].

4 Maintenance prédictive basée sur les données

La maintenance prédictive englobe un ensemble de méthodes visant à anticiper les états futurs et les dommages potentiels des systèmes industriels [3]. En effet, les équipements mécaniques modernes sont généralement instrumentés de capteurs permettant de suivre en temps réel plusieurs paramètres de fonctionnement tels que les vibrations, la température, la pression ou la consommation énergétique [3]. Ces données, accumulées en grande quantité, constituent une source précieuse pour évaluer l'état de santé et la durée de vie utile restante (RUL) à partir de l'historique des contraintes mécaniques (charges, vibrations, températures) et des régimes de fonctionnement actuels. Cependant, la complexité et le volume des données rendent les méthodes d'analyse classiques limitées pour identifier avec précision les comportements annonciateurs de pannes . Il devient alors nécessaire de recourir à des approches capables de modéliser des relations non linéaires, de détecter des schémas cachés et de s'adapter à la variabilité des conditions de fonctionnement. C'est dans ce contexte que le Machine Learning s'impose comme une solution prometteuse pour une maintenance prédictive intelligente. En s'appuyant sur l'apprentissage automatique

à partir des données historiques et temps réel, ces techniques permettent de prédire l'apparition de défaillances, d'estimer la RUL et de détecter des situations anormales avant qu'elles n'impactent la production. L'objectif fondamental reste d'optimiser la prise de décision concernant les interventions, en proposant une planification efficace des tâches de maintenance tout en maîtrisant les coûts. L'intégration du Machine Learning dans les systèmes mécaniques marque ainsi une évolution vers des ensembles mécaniques intelligents, capables d'améliorer leur performance de manière continue grâce à l'exploitation des données [3]. En effet, les équipements industriels modernes sont généralement instrumentés par des capteurs permettant de suivre en temps réel plusieurs paramètres de fonctionnement tels que les vibrations, la température, la pression ou encore la consommation énergétique. Ces données, accumulées en grande quantité au fil du temps, constituent une source d'information précieuse pour l'évaluation de l'état de fiabilité des machines et l'anticipation des défaillances potentielles [3]. Cependant, la complexité et le volume de ces données rendent les méthodes d'analyse classiques limitées pour identifier avec précision les comportements annonciateurs de pannes. Il devient alors nécessaire de recourir à des approches capables de modéliser des relations non linéaires, de détecter des schémas cachés et de s'adapter à la variabilité des conditions de fonctionnement des systèmes industriels. C'est dans ce contexte que le **Machine Learning** s'impose comme une solution prometteuse pour la mise en œuvre d'une maintenance prédictive intelligente. En s'appuyant sur l'apprentissage automatique à partir des données historiques et des données en temps réel, ces techniques permettent de construire des modèles capables de prédire l'apparition de défaillances, d'estimer la durée de vie restante des équipements et de détecter des situations anormales avant qu'elles n'impactent la production. L'intégration du Machine Learning dans les environnements industriels marque ainsi une évolution vers des systèmes de production intelligents, capables d'améliorer leur performance de manière continue grâce à l'exploitation des données.

5 APPRENTISSAGE AUTOMATIQUE (ML)

5.1 DEFINITION

La notion de prédiction dans la littérature est souvent associée à l'utilisation d'un modèle d'apprentissage automatique. Le terme apprentissage automatique, ou Machine Learning (ML) en anglais n'est pas un domaine d'étude récent, il a été proposé pour la première fois en 1959 par Arthur Lee Samuel [4]. Depuis, cette stratégie a été largement adoptée dans des domaines très variés tels que la santé publique, l'agriculture, ou encore la finance. L'idée du Machine Learning consiste à programmer des ordinateurs pour optimiser un critère de performance en utilisant des données d'exemple ou une expérience passée [4]. Nous avons un modèle défini jusqu'à certains paramètres, et l'apprentissage est l'exécution d'un programme informatique pour optimiser les paramètres du modèle en utilisant les données d'entraînement ou l'expérience passée. Le modèle peut être prédictif pour faire des prédictions dans le futur, ou descriptif pour acquérir des connaissances à partir des données, ou les deux. L'apprentissage automatique utilise la théorie des statistiques dans la construction de modèles mathématiques, permettent de déterminer les valeurs d'une variable réponse (Y) à partir d'un ensemble de **variables explicatives** (X), également appelées **attributs**, caractéristiques ou encore features [4]. Chaque exemplaire x de l'ensemble X est appelé **instance** et correspond à l'ensemble des valeurs associées aux attributs. Dans le cadre de la maintenance prédictive, la variable réponse Y peut être continue : le but sera alors de prédire la valeur exacte de cette variable par exemple la mesure de la durée de vie utile restante (RUL) [4]. En plus, la variable Y peut être une valeur booléenne indiquant l'état d'un équipement (normal ou défaillant).

5.2 TYPES APPRENTISSAGE AUTOMATIQUE

Il existe diverses approches pour apprendre automatiquement à partir des données, adaptées aux différents problèmes à résoudre et aux types de données disponibles. La figure 1.3 offre un aperçu des types les plus courants d'apprentissage automatique.

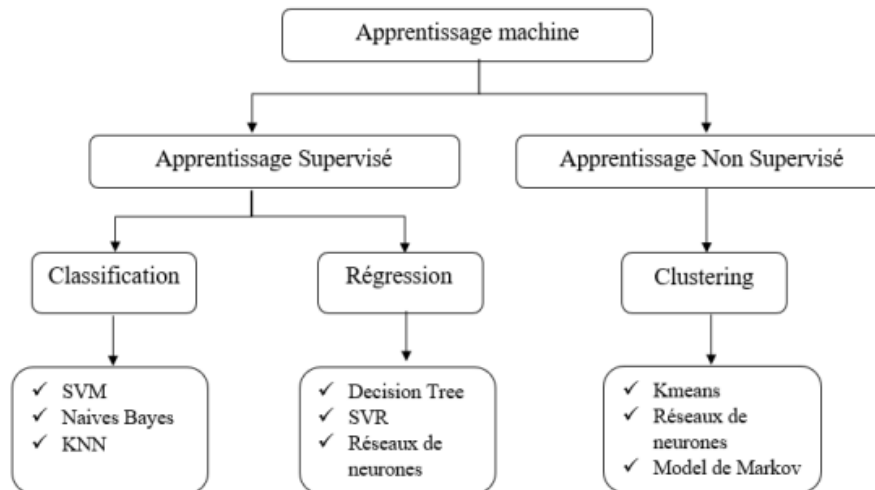


FIGURE 1.3 – Les types d'apprentissage automatique

D'après la figure 1.3, le ML est divisé en deux catégories d'approches que sont l'apprentissage supervisé et l'apprentissage non supervisé, chacune d'elles détaillée ci-après.

5.2.1 APPRENTISSAGE SUPERVISE

L'apprentissage supervisé constitue l'une des tâches les plus simples et les plus répandues en apprentissage automatique. Il repose sur l'utilisation de données d'entraînement étiquetées, c'est-à-dire des données pour lesquelles les résultats attendus sont connus à l'avance [5]. En considérant un ensemble de N entrées x_i accompagnées de leurs sorties cibles correspondantes, on dispose ainsi d'un jeu de données structuré permettant d'entraîner un modèle :

$$D = \{x_i, y_i\}_{i \in [1, N]}$$

Dans le cadre de la maintenance prédictive, l'objectif consiste à entraîner un modèle d'apprentissage automatique capable de prédire l'état futur d'un équipement à partir de données issues de capteurs, même lorsque ces données ne sont pas étiquetées [5]. L'apprentissage repose sur un ensemble de données historiques dans lesquelles les états des machines ou les événements de défaillance sont connus a priori, servant ainsi de référence pour l'entraînement du modèle [5].

La majorité des algorithmes d'apprentissage supervisé fonctionnent en ajustant leurs paramètres de manière à minimiser une fonction de coût, qui mesure l'écart entre les prédictions

du modèle et les valeurs réelles observées. Dans le contexte industriel, cette optimisation permet d'améliorer la précision des prédictions liées à des indicateurs tels que la durée de vie utile restante (RUL) ou la probabilité de défaillance [5].

Les sorties du modèle peuvent correspondre à différentes formes d'indications, telles que la classification de l'état d'un équipement (normal ou défaillant) ou l'estimation d'une variable continue comme la RUL. Ainsi, ces algorithmes cherchent à apprendre une fonction de prédiction capable de capturer les relations complexes entre les données d'entrée issues des capteurs et les comportements de dégradation des machines, afin de faciliter la prise de décision en matière de maintenance [3].

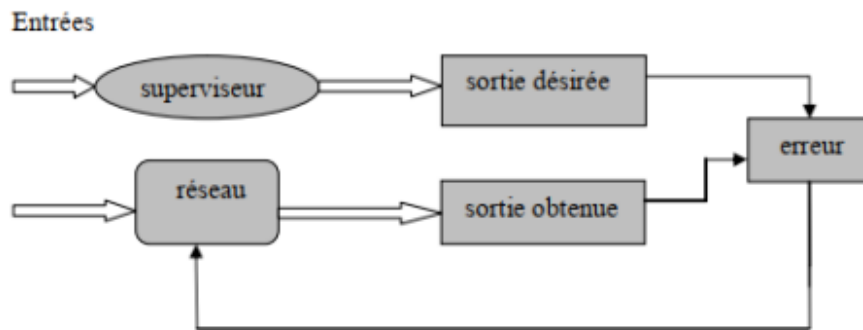


FIGURE 1.4 – Illustration de l'apprentissage supervisé [3]

Les méthodes d'apprentissage supervisé se divisent en deux classes principales en fonction du type de tâches ML qu'elles visent à résoudre.

1. CLASSIFICATION

L'apprentissage supervisé par classification consiste à associer à chaque instance un label appartenant à un ensemble fini de classes discrètes [6]. Dans un contexte industriel, ces classes peuvent représenter, par exemple, l'état d'un équipement (normal ou défaillant) ou différents types de pannes. Le modèle apprend à partir d'un ensemble de données étiquetées noté :

$$D = \{(x_i, y_i)\}_{i=1}^N$$

où x_i représente les caractéristiques extraites des données des capteurs et y_i la classe associée à l'état de l'équipement.

Les problèmes de classification peuvent se décliner en classification binaire ou en classification multi-classes [6]. La classification binaire est utilisée lorsque deux états sont

distingués, tels que le fonctionnement normal et la défaillance d'un équipement [6]. Mathématiquement, pour une classification binaire, on cherche souvent à estimer la probabilité d'appartenance à une classe via une fonction d'activation, telle que la fonction sigmoïde :

$$P(y_i = 1|x_i) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_{i1} + \dots + \beta_n x_{in})}}$$

- x_{ij} représente les **données d'entrée** (caractéristiques capteurs) de l'instance i .
- β_j sont les **poids (coefficients)** que le modèle doit apprendre pour minimiser l'erreur de prédiction.
- La fonction $\frac{1}{1+e^{-z}}$ est la fonction d'activation qui **transforme n'importe quelle valeur en un résultat entre 0 et 1**, interprétable comme une probabilité.

En revanche, la classification multi-classes permet d'identifier plusieurs types de pannes, ce qui est particulièrement utile lorsque les systèmes industriels sont sujets à divers modes de défaillance. Dans certains cas plus complexes, des approches multi-étiquettes peuvent être envisagées lorsque plusieurs anomalies peuvent survenir simultanément [6].

2. REGRESSION

La régression constitue une approche d'apprentissage supervisé largement utilisée pour prédire des variables continues à partir de données d'entrée [6]. Elle repose sur l'établissement d'une relation mathématique entre les variables explicatives et la variable cible, permettant ainsi d'effectuer des estimations fiables sur de nouvelles données. Cette relation peut être formalisée de manière générale comme suit :

$$y = f(x, \theta) + \varepsilon$$

où :

- y représente la **variable dépendante** (par exemple, la durée de vie utile restante ou l'état de santé du système mécanique) ;
- f est la **fonction** (linéaire ou non linéaire) modélisant le comportement du système ;
- x désigne la **variable indépendante** (caractéristiques extraites des données des capteurs : vibrations, températures, pression, etc.) ;
- θ sont les **paramètres inconnus** du modèle, à estimer à partir des données d'apprentissage ;
- ε est le **terme d'erreur**, capturant les bruits de mesure et les écarts non modélisés.

Parmi les techniques les plus courantes figurent la régression linéaire, qui suppose une relation linéaire entre x et y , ainsi que d'autres méthodes plus avancées adaptées à des relations non linéaires. L'objectif principal de la régression est de minimiser l'écart entre les valeurs prédites et les valeurs réelles (généralement en minimisant une fonction de coût liée à ε), afin de construire un modèle capable de généraliser efficacement [6].

Dans le cadre de la maintenance prédictive des systèmes mécaniques, les approches de régression jouent un rôle essentiel, notamment pour l'estimation de la durée de vie utile restante (RUL) des équipements [6]. En effet, les défaillances des systèmes mécaniques sont souvent précédées d'une phase de dégradation progressive, observable à travers l'évolution des données issues des capteurs. La modélisation de ces données, souvent sous forme de séries temporelles, permet de suivre cette dégradation et d'estimer le moment où le système atteindra un état critique. Ainsi, la prédiction de la RUL constitue un outil clé pour anticiper les pannes, optimiser la planification des interventions et améliorer la fiabilité globale des ensembles mécaniques [6].

5.2.2 APPRENTISSAGE NON SUPERVISE

Dans le contexte de l'apprentissage non supervisé, les données exploitées ne sont pas étiquetées, c'est-à-dire qu'elles ne disposent pas de sorties ou de classes prédéfinies [7]. Le modèle est ainsi entraîné à partir d'un ensemble de données d'entrée uniquement, noté :

$$D = \{x_i\}_{i=1}^N$$

où aucune information explicite sur les sorties n'est disponible. L'objectif est alors d'analyser ces données afin d'identifier des structures ou des relations sous-jacentes.

Dans ce contexte, les algorithmes cherchent à organiser les données en groupes homogènes en se basant sur leur similarité. Les méthodes de regroupement, ou *clustering*, permettent ainsi de partitionner les données en différentes classes implicites, de sorte que les éléments appartenant à un même groupe présentent des caractéristiques similaires, tandis que ceux appartenant à des groupes différents sont distincts [7]. Contrairement à l'apprentissage supervisé, ce processus ne repose pas sur des annotations préalablement définies par un

expert, mais uniquement sur les propriétés intrinsèques et les variations observées dans les données [7].

Dans un cadre de maintenance prédictive industrielle, ces approches sont particulièrement utiles pour identifier des modes de fonctionnement des équipements, détecter des comportements anormaux et analyser les données issues des capteurs sans connaissance préalable des états de défaillance [7].

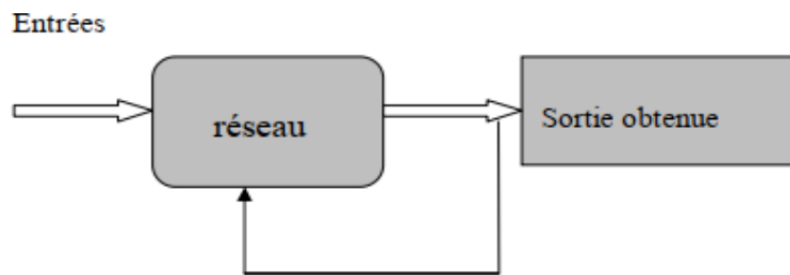


FIGURE 1.5 – Illustration de l'apprentissage non supervisé [3]

1. CLUSTERING (REGROUPEMENT)

Les méthodes de clustering constituent des techniques d'apprentissage automatique non supervisé visant à identifier des structures de similarité au sein d'un ensemble de données [7]. Leur objectif principal est de regrouper les observations en différents ensembles homogènes, appelés clusters, de manière à ce que les éléments appartenant à un même groupe présentent des caractéristiques similaires, tandis que ceux appartenant à des groupes distincts soient significativement différents. Ce regroupement s'effectue uniquement sur la base des propriétés intrinsèques des données, sans recourir à des étiquettes ou à une connaissance préalable des relations entre les variables [7].

Ces approches sont largement utilisées dans divers domaines tels que la bioinformatique, la reconnaissance de formes, la segmentation de marché, l'analyse des réseaux sociaux ou encore la vision par ordinateur. Dans le contexte industriel, et plus particulièrement en maintenance prédictive, elles permettent d'identifier différents modes de fonctionnement des équipements, de détecter des comportements anormaux et de mieux comprendre la dynamique des systèmes à partir des données issues des capteurs [7].

6 Deep Learning pour la maintenance prédictive

6.1 Réseaux récurrents et mécanismes d'attention

6.1.0.1 Long Short-Term Memory(LSTM) Un LSTM classique traite la séquence temporelle du début vers la fin. À chaque pas de temps, il produit un seul état caché à partir de la cellule mémoire et des trois portes (entrée, oubli, sortie), ce qui permet de conserver des dépendances lointaines sans atténuation du gradient [8]. Sur un système mécanique (vibrations, températures, pressions), le LSTM apprend les tendances de dégradation (usure, jeu, fissuration) à partir de fenêtres glissantes de capteurs. L'entraînement supervisé est possible en temps réel car la prédiction n'utilise que le passé [8]. Il capture efficacement les motifs antérieurs à chaque instant, offrant un bon compromis entre précision de la durée de vie restante (RUL) et latence, sans nécessiter la séquence future [8].

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

où :

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f), \quad i_t = \sigma(W_i[h_{t-1}, x_t] + b_i), \quad \tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$$

- x_t : entrée à l'instant t (dimension d_{in})
- h_{t-1} : état caché précédent (dimension d_h)
- c_t : état cellule mis à jour (dimension d_h)
- f_t : porte d'oubli (vecteur de dimension d_h)
- i_t : porte d'entrée (vecteur de dimension d_h)
- $\tilde{c}_t$: candidat cellule (vecteur de dimension d_h)
- W_f, W_i, W_c : matrices de poids (dimension $d_h \times (d_h + d_{\text{in}})$)
- b_f, b_i, b_c : biais (dimension d_h)
- σ : fonction sigmoïde, $\sigma(z) = 1/(1 + e^{-z})$
- $\odot$: produit élément par élément (Hadamard)

6.1.0.2 BiLSTM : Un BiLSTM associe un LSTM forward et un LSTM backward. À chaque pas de temps, il produit deux états cachés (avant et arrière) agrégés pour la sortie, ce qui permet d'utiliser simultanément le contexte passé et futur [9]. Sur un système

mécanique, cela enrichit la détection des motifs de dégradation : par exemple, un pic de vibration suivi d'une tendance douce est interprété différemment que s'il est suivi d'un retour abrupt. On peut y ajouter un mécanisme d'attention. L'entraînement supervisé utilise des fenêtres glissantes. Le BiLSTM améliore la précision du RUL nécessite la séquence complète (calcul offline ou post-analyse) et un coût de calcul plus élevé [9].

$$h_t = [\overrightarrow{h_t}; \overleftarrow{h_t}]$$

où :

$$\overrightarrow{h_t} = \text{LSTM}_{\text{fwd}}(x_t, \overrightarrow{h_{t-1}}, \overrightarrow{c_{t-1}}), \quad \overleftarrow{h_t} = \text{LSTM}_{\text{bwd}}(x_t, \overleftarrow{h_{t+1}}, \overleftarrow{c_{t+1}})$$

- $\overrightarrow{h_t}$: état caché du LSTM forward à l'instant t (dimension d_h)
- $\overleftarrow{h_t}$: état caché du LSTM backward à l'instant t (dimension d_h)
- h_t : état caché final du BiLSTM à l'instant t (dimension $2d_h$)
- $\overrightarrow{h_{t-1}}, \overrightarrow{c_{t-1}}$: état caché et état cellule forward précédents
- $\overleftarrow{h_{t+1}}, \overleftarrow{c_{t+1}}$: état caché et état cellule backward suivants
- LSTM_{fwd} : application des équations LSTM en parcourant la séquence de $t = 1$ à T
- LSTM_{bwd} : application des équations LSTM en parcourant la séquence de $t = T$ à 1
- $[\cdot; \cdot]$: opération de concaténation (fusion des deux vecteurs)

6.1.1 Architectures hybrides

L'architecture hybride CNN 1D – LSTM combine deux types de réseaux de neurones complémentaires pour l'analyse des signaux temporels. Le CNN 1D agit comme un extracteur de caractéristiques locales : ses filtres convolutifs détectent automatiquement des motifs pertinents dans le signal brut tout en réduisant le bruit [8]. Ces représentations sont ensuite transmises au LSTM, qui capture les dépendances temporelles à long terme grâce à son mécanisme à portes. Cette complémentarité permet au modèle de combiner la puissance d'extraction locale du CNN avec la mémoire séquentielle du LSTM, offrant ainsi une analyse riche et robuste des séries temporelles [8].

En maintenance prédictive, ce modèle est appliqué aux signaux issus de capteurs (vibrations, température, courant). Les données sont segmentées en fenêtres temporelles glissantes, puis le CNN 1D en extrait les signatures caractéristiques, tandis que le LSTM analyse leur évolution dans le temps pour détecter une tendance à la dégradation [8]. Le modèle peut ainsi réaliser la détection d'anomalies, la classification de défauts ou la prédiction de la durée de vie résiduelle (RUL) d'un équipement, permettant de réduire les arrêts non planifiés et d'optimiser les interventions de maintenance [8].

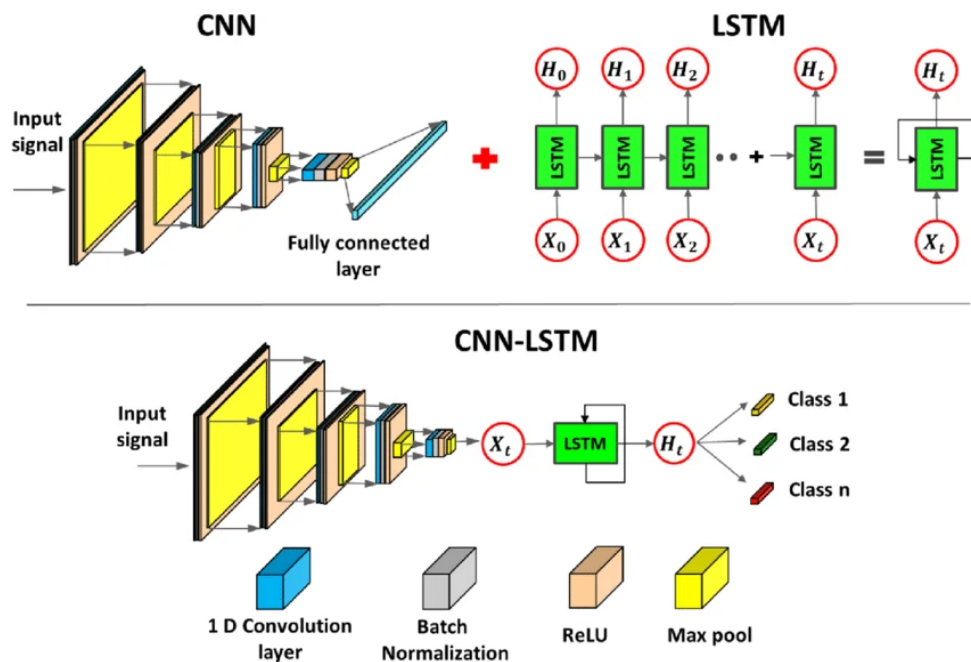


FIGURE 1.6 – Architecture hybride CNN 1D – LSTM pour la classification des signaux — du CNN extracteur de caractéristiques au LSTM modélisateur de dépendances temporelles

7 Conclusion

Ce chapitre a posé les fondations théoriques de notre projet de pipeline MLOps pour la maintenance prédictive des systèmes mécaniques. Nous avons défini le contexte de la maintenance, identifié les limites des stratégies correctives et préventives, et montré comment l'intelligence artificielle permet d'évoluer vers une anticipation intelligente des défaillances. Cette démarche nous a permis de bien cerner les forces et les faiblesses de chaque architecture pour le traitement des données de capteurs. Cette base théorique nous permet désormais d'aborder concrètement la conception du système. Dans le prochain

chapitre, nous explorerons en détail le jeu de données C-MAPSS, définirons l'architecture globale du pipeline MLOps et décrirons chaque étape du flux de données, de l'ingestion jusqu'au le déploiement de la modèle.

Chapitre 2

Analyse et conception

Sommaire

1	Introduction	18
2	Présentation du système étudié	18
2.1	Description du système étudié	18
2.2	DESCRIPTION DE LA BASE DONNEES	20
3	Conception globale du système	21
3.1	Architecture générale du pipeline MLOps	21
3.2	Flux de données	22
4	Modèles de Deep learning pour la maintenance prédictive	28
4.1	Étude des modèles de Deep learning appliqués à la maintenance prédictive	28
4.2	Critères de sélection et évaluation des performances des modèles	33
5	Conclusion	36

1 Introduction

Ce chapitre couvre trois parties essentielles. Premièrement, nous présentons une description détaillée du système étudié (turboréacteur) et du jeu de données C-MAPSS de la NASA, qui sert de base de test. Deuxièmement, nous détaillerons l'architecture globale du pipeline MLOps, montrant le flux complet des données depuis l'ingestion jusqu'au déploiement. Troisièmement, nous effectuerons une évaluation systématique des architectures de Deep Learning en utilisant des métriques standardisées (RMSE, MAE, S-score)

2 Présentation du système étudié

2.1 Description du système étudié

2.1.1 Architecture et fonctionnement du turboréacteur

Le système mécanique considéré dans cette étude est un **turboréacteur à double flux** (turbofan), largement utilisé dans l'aviation commerciale pour sa bonne efficacité énergétique et sa fiabilité. Ce type de moteur est constitué de plusieurs modules principaux interconnectés : une soufflante (fan), des compresseurs basse et haute pression (LPC et HPC), une chambre de combustion, et des turbines basse et haute pression (LPT et HPT) [10]. L'air entrant est d'abord comprimé par la soufflante et les compresseurs, puis mélangé à du carburant dans la chambre de combustion [10]. Les gaz chauds produits entraînent les turbines, qui actionnent les compresseurs et la soufflante, générant ainsi la poussée [10].

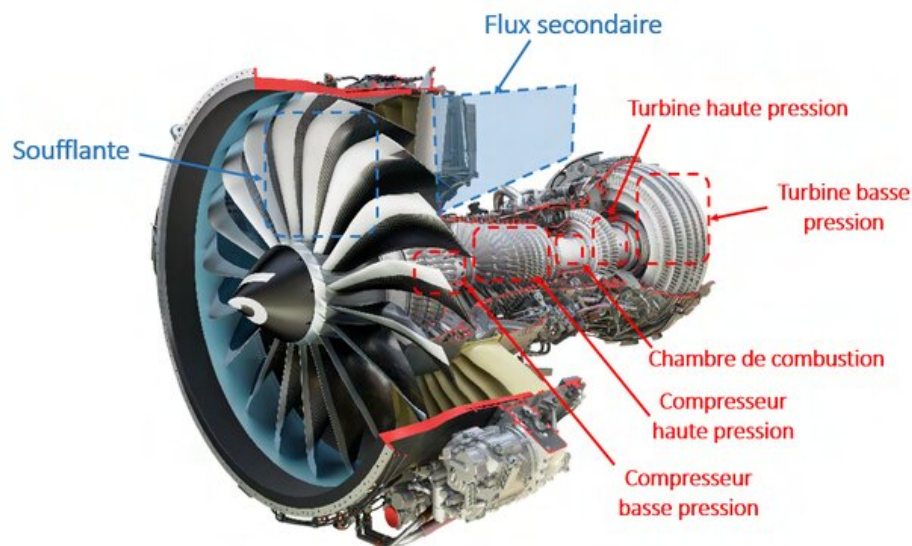


FIGURE 2.1 – Vue d'un turboréacteur double flux

2.1.2 Surveillance par capteurs et prédiction des défaillances

Dans le cadre de la maintenance prédictive, l'état de santé du moteur est suivi en continu à l'aide de capteurs mesurant des grandeurs physiques telles que les températures (entrée, sortie des compresseurs, turbines, échappement), les pressions (différents étages), les régimes de rotation (N_1 , N_2), les débits de carburant, ou encore les niveaux de vibrations. Ces signaux évoluent au cours du temps sous l'effet de l'usure normale (fluage, corrosion, fatigue thermique) ou de défaillances naissantes (dépôts, jeu excessif, déséquilibre) [10].

L'un des phénomènes de dégradation les plus critiques est la perte d'efficacité du compresseur haute pression (HPC), qui se traduit par une augmentation des températures en sortie et une baisse des pressions. D'autres défauts peuvent concerner la soufflante ou la turbine. La progression de ces défauts est généralement lente et non linéaire, rendant difficile la prédiction de la durée de vie utile restante (RUL) par des modèles purement physiques. C'est pourquoi l'exploitation des données historiques issues de capteurs (par exemple via des jeux de référence comme CMAPSS de la NASA) est essentielle pour construire des modèles de pronostic par apprentissage automatique [10].

L'objectif final est d'anticiper les défaillances avant qu'elles ne surviennent, en estimant le nombre de cycles de fonctionnement restants, afin de planifier des opérations de maintenance préventive uniquement lorsque nécessaire, réduisant les coûts et augmentant la

disponibilité des moteurs [10].

2.2 DESCRIPTION DE LA BASE DONNEES

2.2.1 Sous-ensembles du jeu de données C-MAPSS

Le jeu de données C-MAPSS (Commercial Modular Aero-Propulsion System Simulation) est divisé en quatre sous-ensembles – FD001, FD002, FD003 et FD004. Chaque sous-ensemble diffère par le nombre de moteurs simulés, le nombre de conditions opérationnelles et le nombre de modes de panne [10]. Cette gradation de complexité permet d'évaluer la robustesse des modèles de prédiction de durée de vie restante (RUL) face à des environnements de plus en plus réalistes [10]. Les caractéristiques précises de chaque sous-ensemble sont résumées dans le tableau 2.2.1.

Sous-ensemble	Moteurs d'entraînement	Moteurs de test	Conditions opérationnelles
FD001	100	100	1 (niveau de la mer)
FD002	260	259	6
FD003	100	100	1 (niveau de la mer)
FD004	249	248	6

TABLE 2.1 – Composition et caractéristiques des sous-ensembles C-MAPSS

Les quatre sous-ensembles se distinguent principalement par le nombre de conditions opérationnelles et de modes de panne, deux facteurs qui influencent directement la difficulté de la prédiction. FD001 et FD003 ne font intervenir qu'une seule condition de vol, celle du niveau de la mer, tandis que FD002 et FD004 en intègrent six, obtenues par combinaison de l'altitude, du nombre de Mach et de la température d'entrée. Cette diversité de régimes de fonctionnement – du décollage à l'atterrissage – modifie en profondeur la réponse des capteurs, obligeant le modèle à généraliser sur des comportements très variables. Par ailleurs, FD001 et FD002 ne comportent qu'un seul mode de défaillance, la dégradation du compresseur haute pression (HPC), alors que FD003 et FD004 cumulent deux modes simultanés (dégradation du HPC et dégradation du fan) [10]. L'interaction entre ces deux sources d'usure rend la détection des signes précurseurs plus délicate, car leurs effets sur les mesures peuvent se chevaucher ou se masquer mutuellement [10]. Enfin, FD004 se

distingue également par sa taille : avec 61 250 cycles d'entraînement, il offre une base de données substantielle pour les modèles d'apprentissage profond, mais impose en retour des contraintes accrues en mémoire de calcul et en temps d'entraînement. En conséquence, la littérature scientifique utilise fréquemment FD004 comme banc d'essai ultime pour évaluer la capacité de généralisation des méthodes de pronostic, les approches performantes sur les sous-ensembles plus simples (FD001, FD002, FD003) échouant régulièrement sur cette configuration réaliste et complexe [10].

3 Conception globale du système

3.1 Architecture générale du pipeline MLOps

MLOps signifie Machine Learning Operations (opérations d'apprentissage automatique). Cette discipline se concentre sur la rationalisation du processus de déploiement des modèles d'apprentissage automatique en production, puis sur leur maintenance et leur surveillance [11]. MLOps est une fonction collaborative, souvent composée de data scientists, d'ingénieurs ML et d'ingénieurs DevOps. Le terme MLOps est un composé de deux domaines différents : l'apprentissage automatique (Machine Learning) et le DevOps issu de l'ingénierie logicielle [11].

Les MLOps peuvent tout englober, du pipeline de données à la production de modèles d'apprentissage automatique. Dans certains contextes, la mise en œuvre de MLOps ne concerne que le déploiement du modèle, mais d'autres entreprises l'appliquent à de nombreuses étapes du cycle de vie du développement : analyse exploratoire des données (EDA), prétraitement des données, entraînement du modèle, etc [11].

Le terme **pipeline** est généralement utilisé pour décrire la séquence indépendante d'étapes organisées ensemble pour réaliser une tâche [11]. Cette tâche peut relever de l'apprentissage automatique ou non. Un pipeline d'apprentissage automatique est un moyen de contrôler et d'automatiser le flux de travail nécessaire à la production d'un modèle. Il se compose de plusieurs étapes séquentielles allant de l'extraction et du prétraitement des données jusqu'à l'entraînement et au déploiement des modèles. Les pipelines d'apprentissage automatique sont itératifs : chaque étape est répétée pour améliorer en permanence

la précision du modèle et atteindre l'objectif [11]. La figure 2.2 schématise cette organisation en pipeline, où les étapes de validation, prétraitement, entraînement et déploiement sont orchestrées de manière continue avec des boucles de retour du modèle.

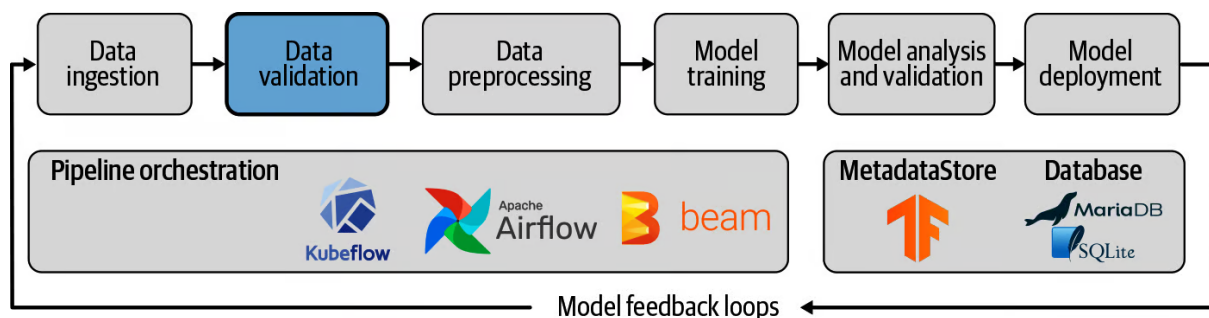


FIGURE 2.2 – Architecture générique d'un pipeline MLOps : de l'ingestion des données au déploiement du modèle avec boucles de rétroaction

3.2 Flux de données

Le flux de données représente le cœur opérationnel du pipeline MLOps. Il décrit le parcours complet des données depuis leur source jusqu'à la génération de prédictions et à la mise en place des boucles de rétroaction [11]. Ce flux est orchestré de manière à assurer une transformation progressive des données brutes en insights exploitables pour la maintenance prédictive d'un système mécanique. La figure 2.3 illustre cette architecture complète, en mettant en avant les différentes étapes et les mécanismes de feedback qui garantissent l'amélioration continue du modèle.

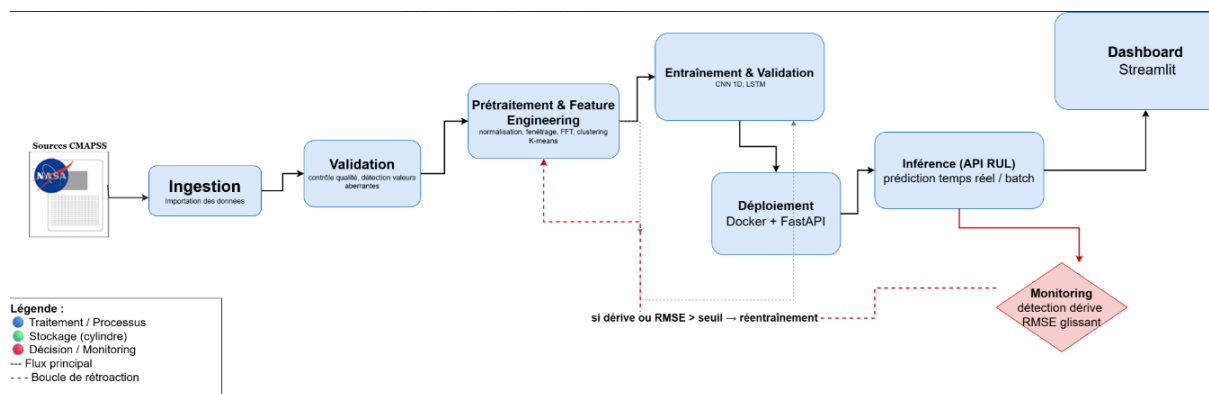


FIGURE 2.3 – Flux de données du système de maintenance prédictive (C-MAPSS)

1. INGESTION DES DONNÉES

L'étape d'ingestion marque le début du pipeline. Les données proviennent donc du dataset C-MAPSS, qui contient des séries temporelles issues de capteurs embarqués sur des turboréacteurs aéronautiques, capturant des paramètres critiques tels que les températures, les pressions, les régimes de rotation (N1, N2) et les débits de carburant.

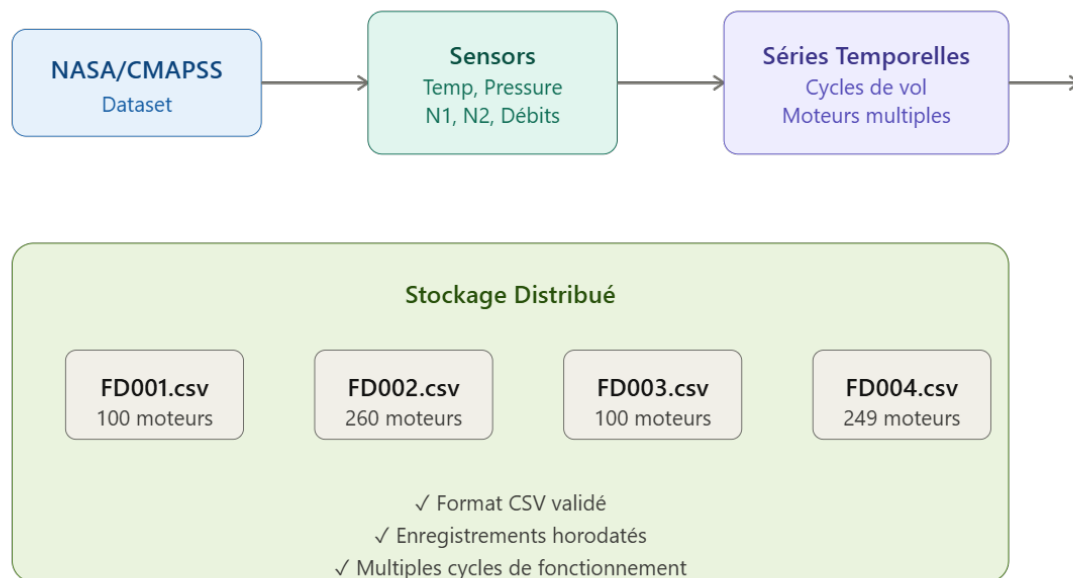


FIGURE 2.4 – Structure et organisation du jeu de données C-MAPSS (NASA)

La Figure 2.4 résume le processus d'ingestion et de structuration initiale des données. Elle met en évidence la provenance des mesures (NASA/C-MAPSS), les paramètres critiques surveillés (température, pression, N1, N2, débits), la nature temporelle et multi-moteur des signaux, ainsi que la segmentation du dataset en quatre fichiers CSV correspondant aux différentes conditions opératoires et modes de défaillance simulés.

2. VALIDATION ET CONTRÔLE DE QUALITÉ

Immédiatement après l'ingestion, une étape de validation est mise en place pour garantir l'intégrité et la cohérence des données. Cette phase critique vérifie plusieurs aspects essentiels : l'absence de valeurs manquantes, la conformité des types de données, la détection d'anomalies évidentes et la vérification des plages de valeurs attendues pour chaque capteur.

Durant le contrôle de qualité, des règles métier spécifiques au contexte aéronautique sont appliquées. Par exemple, il est vérifié que les régimes de rotation (N1, N2) restent dans

des plages physiquement possibles et que les températures ne dépassent pas les seuils de saturation du système. Les capteurs défaillants sont identifiés et marqués pour possible correction ou exclusion. Des statistiques descriptives (moyenne, écart-type, valeurs extrêmes) sont calculées pour chaque variable. La Figure 2.4 illustre les différentes vérifications effectuées pour assurer la qualité des données avant leur utilisation dans le pipeline d'apprentissage.

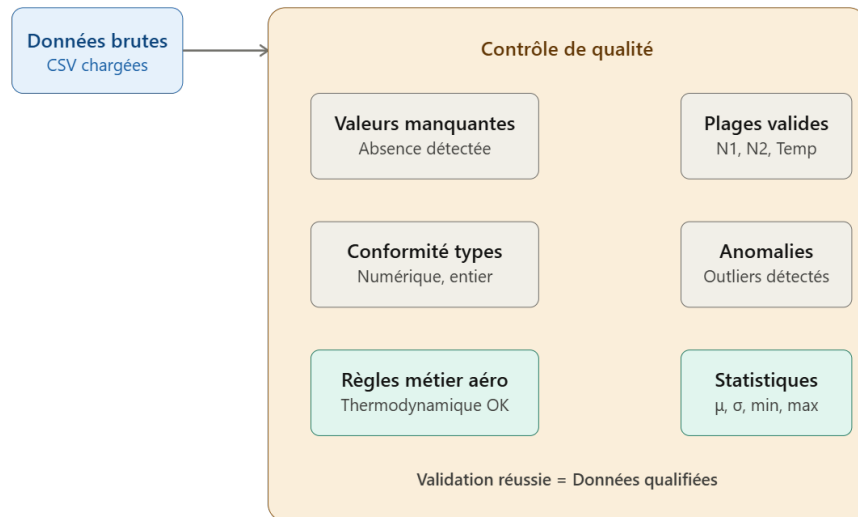


FIGURE 2.5 – Diagramme des vérifications de validation et contrôle de qualité

3. PRÉTRAITEMENT ET EXTRACTION DE CARACTÉRISTIQUES

Le prétraitement constitue une étape fondamentale où les données brutes sont transformées en représentations plus appropriées pour l'apprentissage. Cette phase comprend la normalisation des variables, la gestion des valeurs manquantes et l'extraction de caractéristiques pertinentes.

La normalisation est cruciale car les capteurs possèdent des échelles différentes. Une température varie de quelques degrés Celsius, tandis qu'une pression s'exprime en unités très différentes. L'application de techniques de normalisation (standardisation, normalisation min-max) garantit que tous les features contribuent équitablement au processus d'apprentissage.

L'extraction de caractéristiques est effectuée par clustering K-means pour identifier les régimes de fonctionnement distincts, suivi par des techniques de Feature Engineering : calcul de statistiques locales (moyenne, variance, tendance sur fenêtres glissantes), détection de motifs de dégradation et génération de features basées sur des relations physiques entre capteurs. Pour chaque moteur, une représentation temporelle glissante est construite, où chaque vecteur contient l'historique des k cycles précédents.

La figure 2.6 suivant illustre comment les données brutes subissent une succession de transformations pour devenir des features structurées et prêtes pour l'entraînement du modèle.

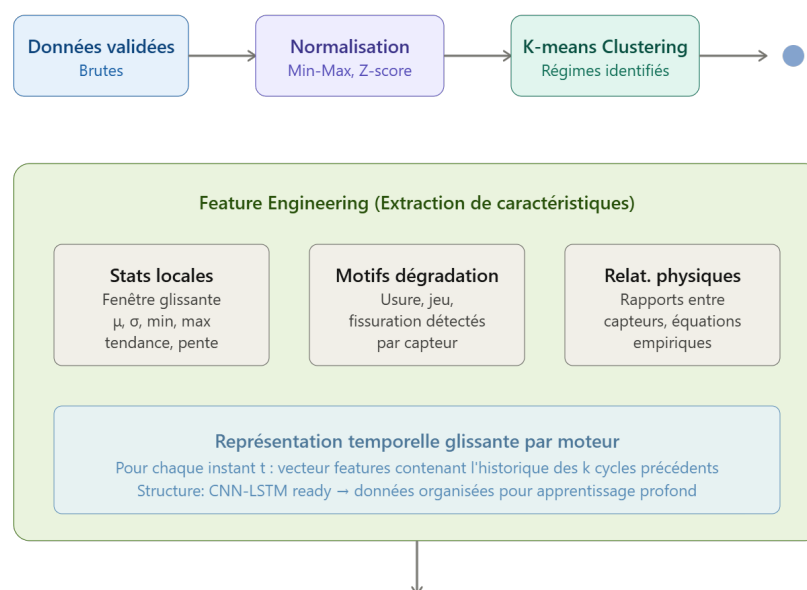


FIGURE 2.6 – Diagramme du prétraitement et extraction de caractéristiques

4. ENTRAÎNEMENT ET VALIDATION DU MODÈLES

Une fois les données prétraitées et les caractéristiques extraites, le pipeline procède à la division des données en ensembles d'entraînement et de validation. Cette scission est effectuée temporellement et par moteur, respectant la chronologie des séries temporelles.

La validation régulière sur un ensemble de test permet de surveiller la généralisation du modèle et de détecter les phénomènes de surapprentissage. Lorsque la performance détectée se dégrade au-delà d'un seuil prédéfini, un signal de réentraînement peut être déclenché automatiquement.

Le diagramme suivant montre comment les données divisées sont utilisées pour entraîner le modèle et valider sa performance en continu.

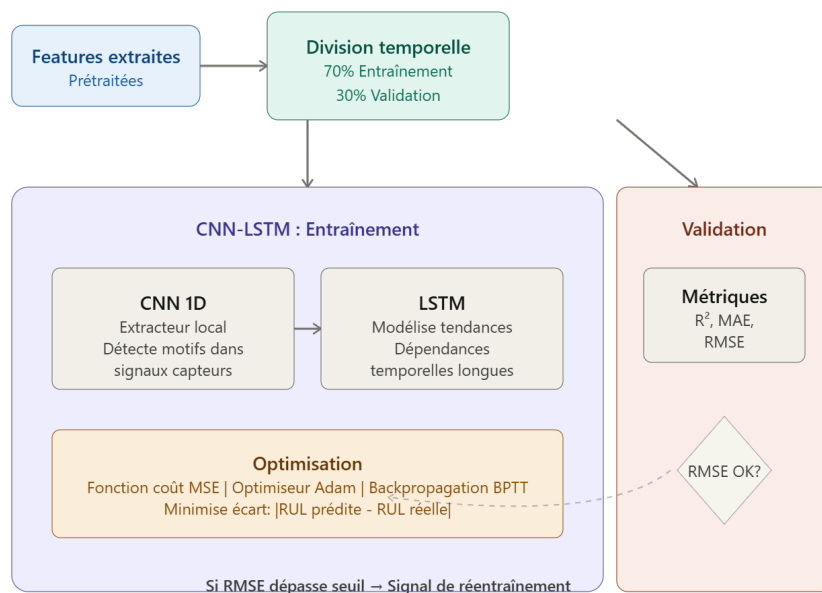


FIGURE 2.7 – Diagramme de l'entraînement et validation du modèle

5. DÉPLOIEMENT ET INFÉRENCE

Une fois le modèle entraîné et validé, il est déployé en environnement de production via une architecture conteneurisée. Cette approche garantit la reproductibilité et la portabilité du modèle sur différentes infrastructures. Une interface de programmation (API) basée sur une framework REST expose le modèle via un endpoint HTTP standardisé, permettant l'intégration aisée avec des systèmes externes.

L'inférence est proposée selon deux modes : un mode batch pour le traitement de larges volumes de données historiques, où plusieurs moteurs peuvent être traités en parallèle optimisant l'utilisation des ressources ; et un mode temps réel pour les prédictions unitaires, où les nouvelles données de capteurs arrivant du système de surveillance sont traitées immédiatement.

Le modèle génère pour chaque moteur une estimation du nombre de cycles restants avant défaillance, associée à un intervalle de confiance. Ces résultats sont communiqués au tableau de bord de suivi pour visualisation de l'état de santé de tous les moteurs.

Le diagramme suivant illustre le pipeline de déploiement et les deux modes d'inférence disponibles pour générer les prédictions.

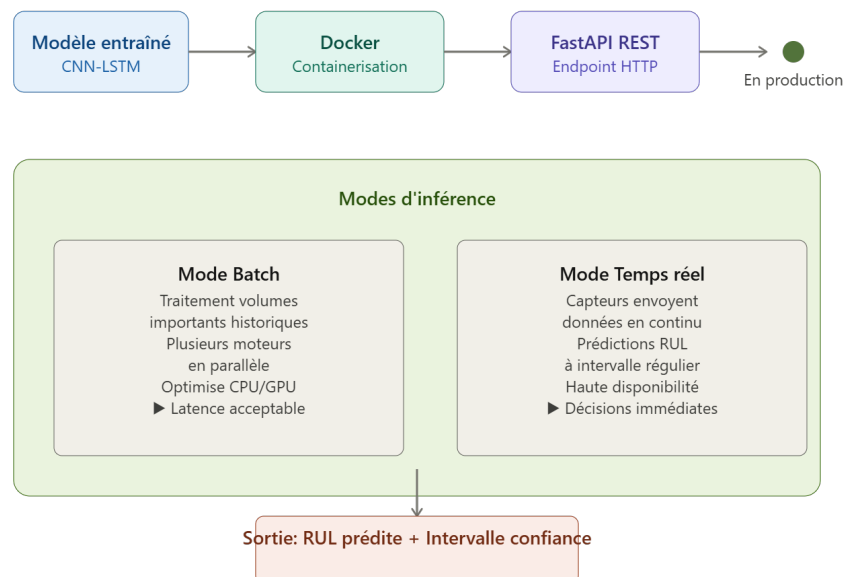


FIGURE 2.8 – Diagramme du déploiement et inférence

4 Modèles de Deep learning pour la maintenance prédictive

La prédiction de la durée de vie utile restante (RUL) d'un système mécanique repose sur l'analyse de séries temporelles multivariées, souvent non linéaires et issues de capteurs.

4.1 Étude des modèles de Deep learning appliqués à la maintenance prédictive

1. LSTM

Le LSTM (Long Short-Term Memory) est un réseau neuronal récurrent conçu pour traiter les séquences temporelles en mémorisant les informations importantes sur de longues périodes [12].

Dans notre projet, Le réseau LSTM constitue le modèle de base pour le traitement des séquences de cycles de vol du dataset C-MAPSS. Grâce à ses portes logiques (entrée, oubli, sortie), il est capable de mémoriser les informations sur de longues séquences temporelles sans subir le problème de la disparition du gradient [12]. Il modélise l'évolution de la dégradation du turboréacteur cycle par cycle, en capturant la tendance continue de l'usure mécanique à partir des fenêtres de données des 21 capteurs [12]. La Figure 2.10 détaille l'architecture complète du réseau LSTM implémenté dans notre pipeline, depuis la fenêtre glissante appliquée aux 14 capteurs retenus jusqu'à l'estimation finale de la durée de vie restante, en passant par l'empilement des couches récurrentes et la régularisation par Dropout.

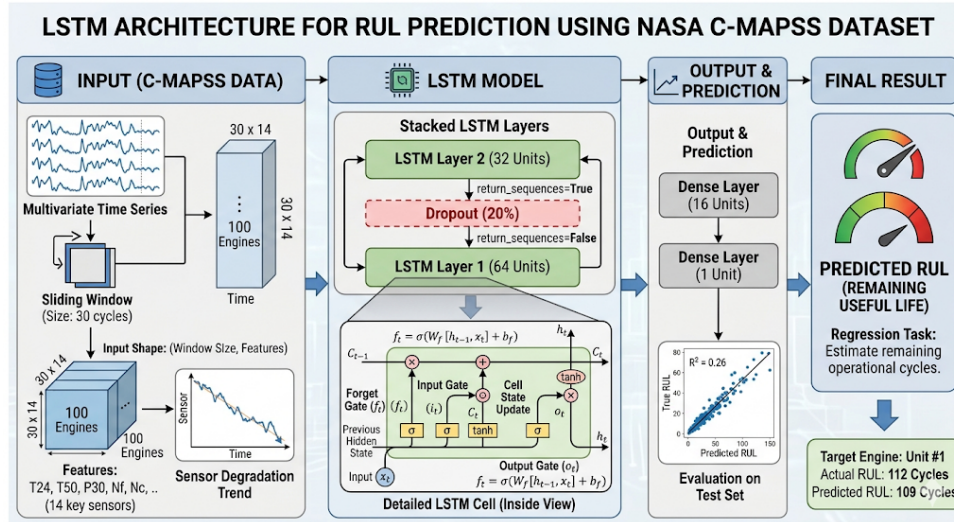


FIGURE 2.9 – Architecture du modèle LSTM pour la prédiction de la durée de vie restante (RUL) à partir des données C-MAPSS

2. CNN-1D

Le CNN-1D (Convolutional Neural Network 1D) est un réseau convolutif qui utilise des filtres glissants pour détecter automatiquement des motifs et des signatures caractéristiques dans les séries temporelles [12]. Dans notre projet, Les réseaux de neurones convolutifs à une dimension (CNN-1D) sont utilisés pour l'extraction de caractéristiques spatiales. En appliquant des filtres sur les fenêtres temporelles des capteurs du C-MAPSS, le CNN-1D identifie automatiquement des corrélations locales entre les différentes grandeurs physiques (par exemple, un pic de température associé à une baisse de pression) et agit comme un filtre réducteur de bruit, préparant les données brutes pour la prédiction [12]. La Figure 2.11 résume le pipeline CNN-1D : fenêtres glissantes normalisées, extraction convolutive, et couches denses produisant une estimation de RUL illustrée à 109 cycles prédits contre 112 cycles réels.

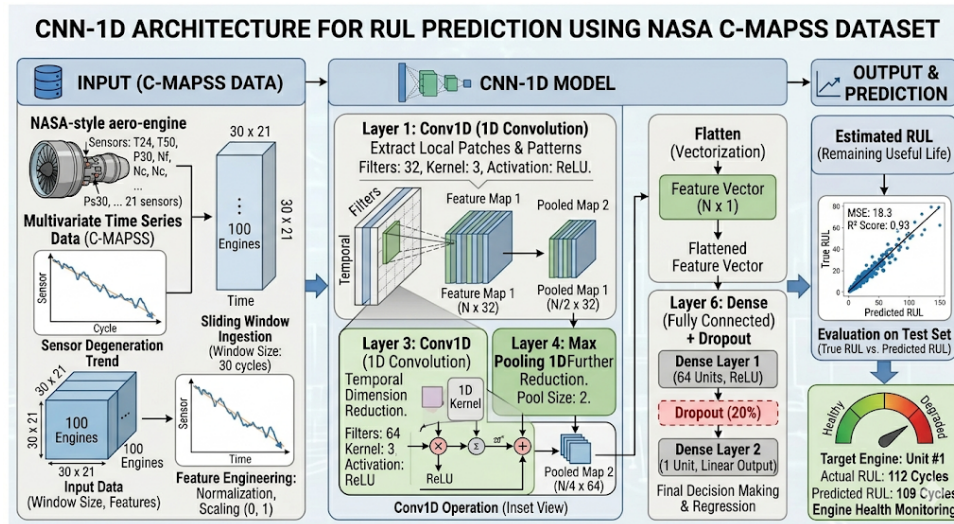


FIGURE 2.10 – Architecture du modèle CNN-1D pour l'estimation de la durée de vie restante (RUL) à partir des données C-MAPSS

3. BILSTM

Le modèle BiLSTM (Bidirectional LSTM) proposé utilise les données normalisées avant l'entraînement. Il se compose de trois couches LSTM bidirectionnelles ainsi que de deux couches denses. L'avantage de l'utilisation d'un LSTM bidirectionnel réside dans sa capacité à traiter les séquences de données d'entrée à la fois dans le sens direct et dans le sens inverse [12].

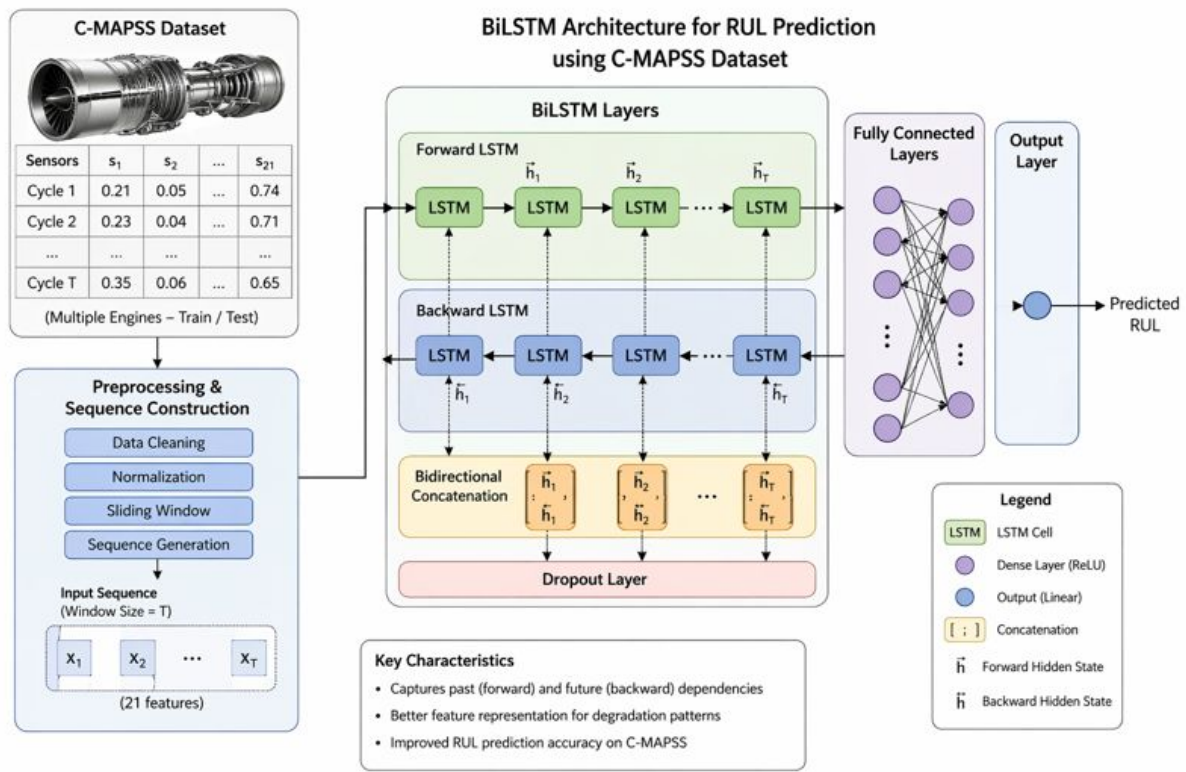


FIGURE 2.11 – Architecture du modèle BiLSTM pour l'estimation de la durée de vie restante (RUL) à partir des données C-MAPSS

4. TRANSFORMER

Le modèle Transformer constitue une architecture de réseaux de neurones conçue pour traiter des données séquentielles à l'aide de mécanismes d'auto-attention (self-attention). Contrairement aux réseaux de neurones récurrents (RNN) classiques, les Transformers gèrent efficacement les dépendances à long terme grâce à leur capacité à traiter les données en parallèle [12]. Cette caractéristique les rend particulièrement adaptés à l'analyse des interactions entre les différentes variables du dataset C-MAPSS en utilisant des têtes d'attention multiples qui apprennent à se concentrer sur différents aspects des données de dégradation du turboréacteur [12].

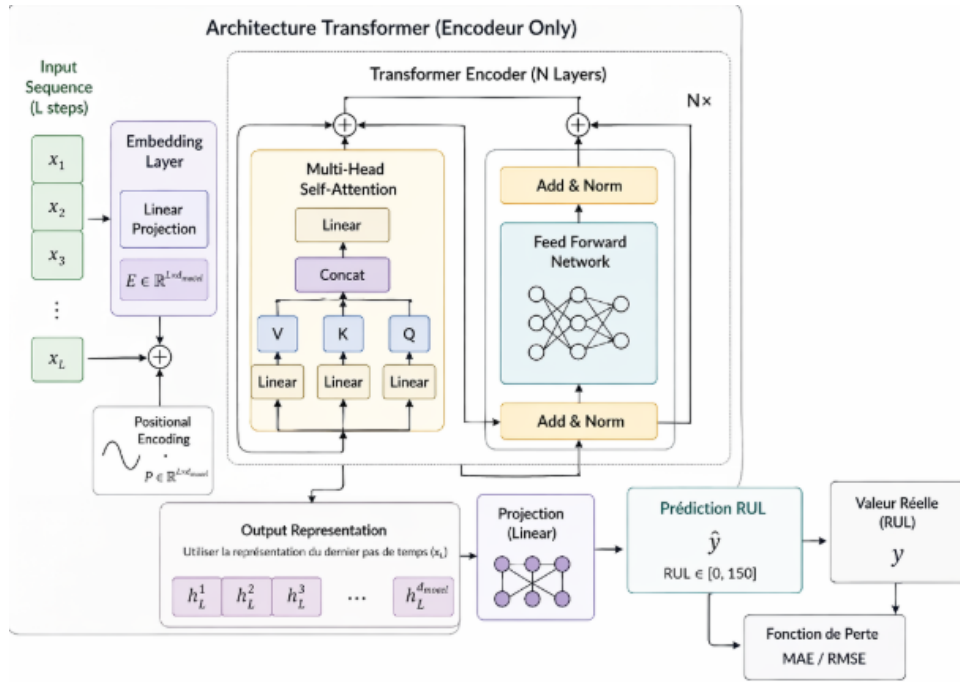


FIGURE 2.12 – Architecture du modèle Transformer pour l'estimation de la durée de vie restante (RUL) à partir des données C-MAPSS

5. CNN-LSTM-TRANSFORMER (HYBRIDE COMPLET)

Cette architecture combine trois étages : le CNN-1D extrait les motifs locaux des signaux, le LSTM modélise les dépendances temporelles, et le Transformer raffine la représentation via attention globale [12]. C'est l'approche complète testée sur C-MAPSS. Le CNN détecte d'abord les signatures de dégradation dans les données brutes des capteurs, le LSTM suit ensuite comment ces motifs évoluent au cours de la vie du moteur, et le Transformer identifie les interactions critiques entre les différentes phases de dégradation pour la prédiction finale du RUL [12]. La Figure 2.14 montre l'architecture CNN-LSTM-Transformer hybride complète : les données brutes (Input Predictors) passent d'abord par deux couches de convolution (1st et 2nd CNN) qui extraient les motifs locaux des signaux, puis les features extraites sont traitées par une pile de couches LSTM qui modélisent les dépendances temporelles, et enfin par un mécanisme Transformer avec attention multi-tête qui raffine la représentation en capturant les relations globales entre les différentes phases de dégradation. Une couche dense entièrement connectée (Fully Connected Dense Layer) transforme ensuite cette représentation enrichie en une prédiction finale du RUL (Remaining Useful Life).

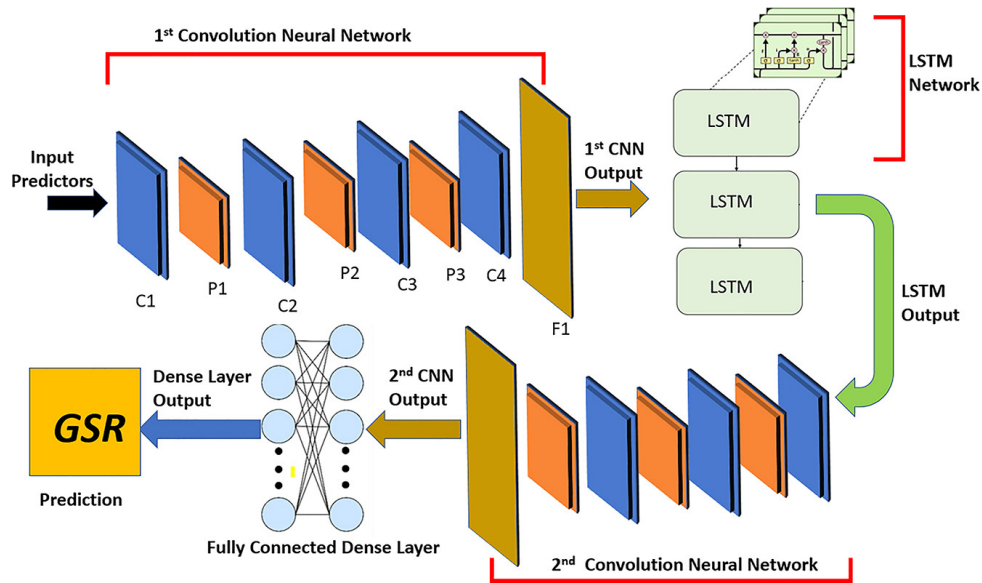


FIGURE 2.13 – Architecture hybride combinant des couches convolutives (CNN), récurrentes (LSTM) et un module Transformer (mécanisme d’attention) pour la prédiction de la durée de vie restante.

4.2 Critères de sélection et évaluation des performances des modèles

4.2.1 Métriques utilisées

Pour évaluer la qualité des prédictions de RUL, deux métriques principales ont été utilisées : le RMSE (Root Mean Square Error) et le MAE (Mean Absolute Error).

1. RMSE (Root Mean Square Error)

Le RMSE mesure l’écart quadratique moyen entre les valeurs réelles et les valeurs prédites [13]. C’est la métrique la plus couramment utilisée pour évaluer les modèles de régression. Elle est définie mathématiquement comme :

$$RMSE = \sqrt{\frac{\sum_{i=1}^N (x_i - \bar{x})^2}{N}}$$

Où :

- x_i représente la valeur réelle observée
- $\bar{x}$ représente la valeur prédite

- N est le nombre total d'échantillons

Plus la valeur de RMSE est faible, meilleure est la performance du modèle. Un RMSE proche de zéro indique que les prédictions sont très proches des valeurs réelles [13].

2. MAE (Mean Absolute Error)

Le MAE mesure l'erreur absolue moyenne entre les valeurs réelles et les valeurs prédites [13]. Contrairement au RMSE qui pénalise davantage les grandes erreurs, le MAE traite toutes les erreurs de façon égale [13].

$$MAE = \frac{1}{N} \sum_{i=1}^N |x_i - \bar{x}|$$

Où :

- x_i représente la valeur réelle observée
- $\bar{x}$ représente la valeur prédite
- N est le nombre total d'échantillons

Le MAE fournit une indication moyenne de l'amplitude des erreurs de prédiction. Un MAE faible indique un modèle fiable [13].

3. F1-score

Le F1-score est la moyenne harmonique entre la précision et le rappel, variant entre 0 (mauvais) et 1 (parfait). Il est particulièrement adapté aux classes déséquilibrées.

$$F1 = 2 \times \frac{\text{précision} \times \text{rappel}}{\text{précision} + \text{rappel}} \quad (2.1)$$

4. ROC-AUC

La ROC-AUC mesure l'aire sous la courbe ROC. Une valeur de 0,5 correspond à un classifieur aléatoire, tandis qu'une valeur proche de 1 indique une excellente séparation entre les classes.

$$AUC = \int_0^1 \text{taux de vrais positifs } d(\text{taux de faux positifs}) \quad (2.2)$$

5. S-score (Scoring Function)

Le S-score est une fonction de scoring spécialisée pour la prédiction de RUL [13]. Cette métrique pénalise davantage les prédictions tardives que les prédictions précoces, car une prédiction tardive peut avoir des conséquences plus graves en termes de sécurité [13].

Formule :

$$S\text{-score} = \frac{1}{n} \sum_{i=1}^n \left(\frac{|RUL_i - \hat{RUL}_i|}{\max(RUL_i)} \right)$$

Où :

- RUL_i : valeur réelle de la durée de vie restante
- $\hat{RUL}_i$: valeur prédite de la durée de vie restante
- n : nombre d'échantillons
- $\max(RUL_i)$: valeur maximale de RUL observée dans le jeu de données

4.2.2 Évaluation des performances des modèles

Une bonne performance de modèle est caractérisée par :

- Un **RMSE faible** : idéalement < 22 cycles pour le dataset C-MAPSS [13].
- Un **MAE faible** : l'erreur moyenne doit être minimisée [13].
- Un **S-score** : Plus la valeur du S-score est proche de zéro plus la performance est meilleure [13].

5 Conclusion

Ce chapitre a présenté une analyse complète du système, depuis la description du turboréacteur et du dataset C-MAPSS jusqu'à l'architecture globale du pipeline MLOps, en passant par l'étude détaillée de chaque architecture de Deep Learning appliquée à la maintenance prédictive. L'évaluation comparative de ces modèles, fondée sur des métriques standardisées telles que le RMSE, le MAE et le S-score, a permis de mesurer leur précision et leur capacité à éviter les prédictions tardives.

Le chapitre suivant détaille la réalisation pratique du pipeline, en couvrant l'implémentation des services applicatifs (API REST, dashboard Streamlit), la conteneurisation Docker, et le déploiement

Chapitre 3

Réalisation et tests

Sommaire

1	Introduction	39
2	Environnement technique et outils	39
2.1	Langage	39
2.2	Bibliothèques	40
2.3	Environnement de développement	42
3	Préparation et prétraitement des données	44
3.1	Prétraitement des données pour la prédiction RUL	44
3.2	Préparation des données pour les modules annexes (anomalies, SHAP))	46
4	Développement des services applicatifs	46
4.1	API REST avec FastAPI	46
4.2	Dashboard interactif avec Streamlit	47
5	Conteneurisation avec Docker	48
5.1	définition et objectifs	48
6	Déploiement sur Hugging Face Spaces	52
6.1	Création du Space Docker et configuration du dépôt	52
6.2	Versionnement et push du code via Git	54
7	Validation des modèles et analyse des résultats	55
7.1	Résultats comparatifs	55
7.2	Analyse approfondie des résultats par modèle	56

8	Conclusion	58
---	----------------------	----

1 Introduction

Ce chapitre expose quatre domaines clés. D'abord, nous détaillerons l'environnement technique et les outils utilisés (Python, PyTorch, FastAPI, Streamlit, Docker). Ensuite, nous couvrirons le prétraitement et la préparation des données, étapes critiques pour éviter les fuites d'informations et garantir la généralisabilité du modèle. Puis, nous développons les services applicatifs (API REST et dashboard interactif). Enfin, nous aborderons la conteneurisation avec Docker et le déploiement sur Hugging Face Spaces.

2 Environnement technique et outils

2.1 Langage

2.1.1 Python

Python est le langage de programmation le mieux adapté à la mise en place d'un pipeline MLOps dédié à la maintenance prédictive d'un système mécanique. Il a été choisi pour sa capacité à couvrir l'ensemble du processus : du prétraitement des séries temporelles issues des capteurs d'un turboréacteur, à l'entraînement de modèles de deep learning (LSTM, CNN-LSTM, Transformer) avec PyTorch, jusqu'au développement d'une API REST avec FastAPI et d'un tableau de bord interactif.



FIGURE 3.1 – logo python

2.2 Bibliothèques

2.2.1 PyTorch

PyTorch est une bibliothèque d'apprentissage profond . Elle a été choisie pour sa flexibilité (exécution dynamique des graphes) et sa facilité de débogage, essentielles pour expérimenter avec des architectures séquentielles (LSTM, CNN-LSTM, Transformer). Dans ce projet, PyTorch a servi à implémenter, entraîner et évaluer les modèles de prédiction du RUL.



FIGURE 3.2 – logo PyTorch

2.2.2 Streamlit

Streamlit est une bibliothèque open source qui transforme des scripts Python en applications web interactives sans nécessiter de connaissances en HTML/CSS/JavaScript. Elle a permis de développer rapidement un tableau de bord multi-pages pour visualiser les prédictions, les alertes et les recommandations techniques. La communication avec l'API via requests est native et le déploiement est simplifié.



FIGURE 3.3 – logo StreamLit

2.2.3 scikit-learn

scikit-learn est la bibliothèque de référence pour l'apprentissage automatique classique en Python. Elle a été utilisée pour la normalisation des données , la réduction de dimension , la détection d'anomalies avec Isolation Forest, et le calcul des métriques (RMSE, MAE) lors de l'évaluation des modèles. Sa compatibilité avec PyTorch facilite l'intégration dans un pipeline hybride.



FIGURE 3.4 – logo scikit-learn

2.2.4 Numpy

NumPy est la bibliothèque fondamentale pour le calcul numérique en Python. Elle introduit les tableaux multidimensionnels et une collection de fonctions mathématiques optimisées. Dans ce projet, NumPy a été utilisé pour manipuler les séries temporelles des capteurs.



FIGURE 3.5 – logo Numpy

2.3 Environnement de développement

2.3.1 Visual Studio Code (VS Code)

VS Code a été l'éditeur de code principal. Il offre une intégration native avec Git, un terminal intégré, des extensions pour Python, Docker, et même une connexion directe aux espaces Hugging Face via Remote SSH. Son débogueur interactif a grandement facilité la mise au point des scripts de prétraitement et des endpoints de l'API.

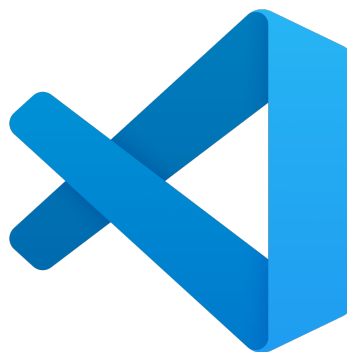


FIGURE 3.6 – logo VSCode

2.3.2 Git

Git est le système de contrôle de version utilisé pour suivre l'évolution du code source, collaborer et surtout pour déployer sur Hugging Face Spaces (chaque git push déclenche un nouveau build).



FIGURE 3.7 – logo Git

2.3.3 Hugging Face Spaces

Hugging Face est une plateforme de déploiement de modèles et d'applications ML. Elle a été choisie pour héberger le pipeline final via un Space de type `Docker`, offrant un déploiement continu par `git push`, un suivi des logs, une gestion automatique du cycle de vie et une URL publique pour la démonstration.



FIGURE 3.8 – Hungging face logo

2.3.4 Docker Desktop

Docker Desktop a permis de construire et tester localement l'image Docker avant le déploiement. Grâce à son interface graphique, il est possible de visualiser les conteneurs en cours d'exécution, d'examiner les logs et de gérer les volumes.



FIGURE 3.9 – logo Docker Desktop

3 Préparation et prétraitement des données

3.1 Prétraitement des données pour la prédiction RUL

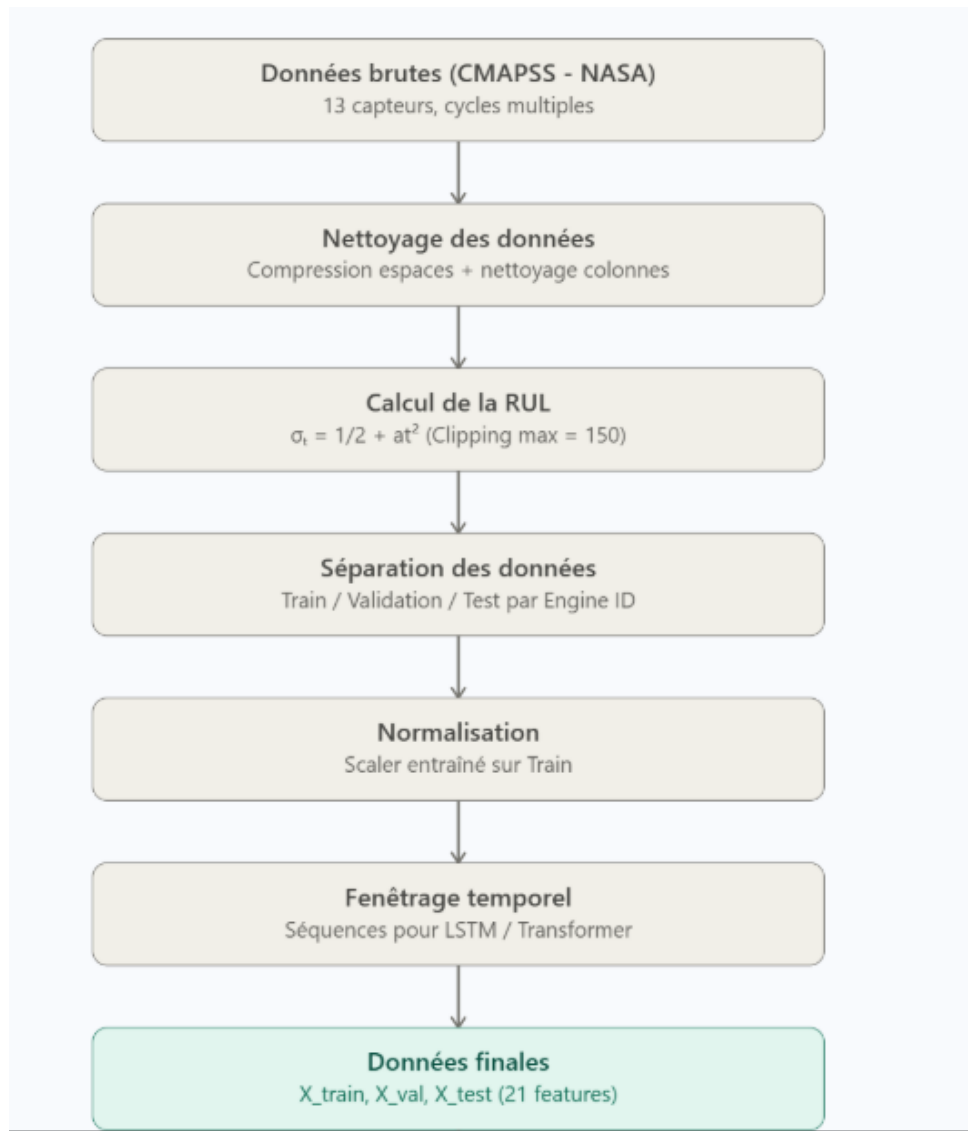


FIGURE 3.10 – Les étapes de prétraitement des données

Normalisation :

La normalisation des données est effectuée à l'aide d'un `StandardScaler` (normalisation Z-score) dont la formule est $z = \frac{x - \mu}{\sigma}$. Ce scaler est ajusté uniquement sur les moteurs de l'ensemble d'entraînement, afin d'éviter toute fuite d'informations provenant des ensembles de validation ou de test (leurs moyennes et écarts-types ne sont pas utilisés). Ensuite, la même transformation est appliquée aux ensembles de validation et de test.

Fenêtrage :

Le modèle LSTM/BiLSTM nécessite une séquence temporelle pour capturer l'évolution de la dégradation du moteur. La taille de la fenêtre est fixée à 50 cycles : le modèle analyse les 50 derniers cycles pour estimer le RUL au cycle présent.

Construction des fenêtres Pour un moteur ayant 100 cycles, on génère les fenêtres suivantes : [cycles 1 à 50], [cycles 2 à 51], ..., jusqu'à [cycles 51 à 100]. Chaque fenêtre est associée à un label : la valeur du RUL correspondant au dernier cycle de la fenêtre.

Division des ensembles et prévention des fuites de données :

La division des ensembles constitue l'étape la plus critique pour éviter toute fuite de données (*data leakage*). Contrairement à une division aléatoire classique, nous séparons les données par **identifiants de moteurs**, car les cycles d'un même moteur sont temporellement liés. Si l'on plaçait le cycle 10 d'un moteur dans l'ensemble d'entraînement et son cycle 11 dans l'ensemble de validation, le modèle apprendrait à reconnaître la signature propre à ce moteur, ce qui fausserait son évaluation. La répartition adoptée est la suivante : 80 % des moteurs pour l'entraînement, 10 % pour la validation et 10 % pour le test interne. Cette répartition garantit que les séquences d'un même moteur ne se retrouvent jamais dans des ensembles différents.

Écrêtage du RUL (RUL Clipping / Piecewise RUL) :

L'écrêtage du RUL (ou *piecewise RUL*) est une pratique standard en PHM (*Prognostics and Health Management*). Elle s'explique par le fait qu'en début de vie, le moteur est considéré comme sain et les capteurs ne montrent aucune dégradation significative, ce qui rend la prédiction d'une valeur de RUL très élevée (par exemple 300 cycles) à la fois difficile et peu pertinente pour la maintenance. La méthode consiste à fixer une limite maximale (généralement 125 ou 150 cycles selon le sous-ensemble FD). Toutes les valeurs de RUL supérieures à ce seuil sont alors ramenées à cette limite, produisant une fonction cible en forme de palier (*plate*) en début de vie, puis linéaire décroissante à mesure que le moteur approche de la panne.

3.2 Préparation des données pour les modules annexes (anomalies, SHAP))

L'objectif est d'entraîner un modèle de détection d'anomalies, en l'occurrence une **forêt d'isolation (Isolation Forest)**, capable de discriminer les fenêtres de données capteurs normales des fenêtres anormales. Pour constituer l'ensemble d'apprentissage dit normal, on extrait les fenêtres correspondant aux cycles de début de vie des moteurs, définis par un RUL strictement supérieur à 100 cycles. À ce stade, aucune dégradation significative n'est encore détectée. Afin de réduire la dimensionnalité et d'extraire des descripteurs pertinents, chaque fenêtre de forme (50 cycles, 21 capteurs) est transformée en un **vecteur de caractéristiques statistiques** comprenant la moyenne, l'écart-type et la dernière valeur pour chaque capteur. Ces attributs résument le comportement temporel du signal et servent d'entrée à l'Isolation Forest.

Parallèlement, le module **SHAP** nécessite une préparation spécifique pour assurer l'explicabilité des prédictions du modèle de deep learning (Transformer). Un **jeu de fond (background dataset)** est constitué par échantillonnage aléatoire de 500 fenêtres représentatives de la distribution d'entraînement. Cet échantillon sert de référence pour calculer l'importance de chaque capteur : SHAP compare la prédiction d'une instance donnée à la prédiction moyenne obtenue sur ce jeu de fond, permettant ainsi d'isoler la contribution individuelle de chaque variable. En complément, un **fichier de métadonnées** assure le mapping entre les indices numériques des capteurs (0 à 20) et leurs libellés explicites, garantissant ainsi une visualisation intelligible des graphiques SHAP.

4 Développement des services applicatifs

4.1 API REST avec FastAPI

4.1.1 Architecture générale et endpoints exposés

L'API est développée avec FastAPI pour ses hautes performances et sa validation automatique via Pydantic. Elle expose les endpoints suivants :

Méthode	Chemin	Paramètres	Description
GET	/health	Aucun	État de santé de l'API.
POST	/predict	window, model_name	Prédiction du RUL .
POST	/anomalies	window, engine_id	Détection d'anomalies (Isolation Forest).
POST	/explain	window, top_k	Explication locale SHAP.
POST	/recommend_tech	engine_id, cycle	Fiche technique d'actions.
POST	/engine_zones	window	Scores d'usure par zone

TABLE 3.1 – Endpoints de l'API REST

4.1.2 Fonctionnement technique

Chargement des modèles : Les modèles (poids PyTorch et scalers) sont préchargés en mémoire au démarrage de l'API (événement `startup`) avec mise en cache, garantissant une latence inférieure à 200 ms.

Formats d'échange : La requête `/predict` attend une matrice `window` ($50 \text{ cycles} \times 24 \text{ capteurs}$) et un booléen `already_scaled`. La réponse fournit le RUL prédit ainsi que des métadonnées (modèle utilisé, taille fenêtre, etc.).

Documentation automatique : Une interface interactive Swagger est disponible sur `/docs` pour tester les endpoints.

4.2 Dashboard interactif avec Streamlit

Le dashboard est développé avec **Streamlit**, il communique avec l'API via la bibliothèque **requests** : chaque page du dashboard envoie des requêtes HTTP aux endpoints décrits précédemment pour obtenir des prédictions, des détections d'anomalies, des explications SHAP ou des recommandations techniques. Les réponses de l'API sont ensuite affichées sous forme de graphiques (évolution des capteurs, RUL estimé), de tableaux d'alertes ou de fiches d'actions de maintenance. Cette architecture découplée permet de faire évoluer indépendamment l'interface utilisateur et les services de prédiction. Le dashboard est organisé en plusieurs pages accessibles via une barre latérale : accueil, détail d'un moteur, alertes en temps réel, historique des prédictions et analyse interactive (drift, anomalies,

SHAP). Les modèles et les scalers restent chargés dans l'API, ce qui allège le dashboard et garantit des temps de réponse rapides.

5 Conteneurisation avec Docker

5.1 définition et objectifs

Docker est une plateforme de conteneurisation qui permet d'empaqueter une application et toutes ses dépendances (bibliothèques, variables d'environnement, fichiers de configuration) dans un conteneur isolé et léger. Contrairement à une machine virtuelle, un conteneur partage le noyau du système d'exploitation hôte, ce qui le rend plus rapide à démarrer et moins consommateur de ressources. L'objectif principal de Docker est d'assurer la reproductibilité et la portabilité de l'environnement d'exécution : une application conteneurisée se comporte de manière identique sur la machine du développeur, sur un serveur de test ou dans le cloud. Dans un projet MLOps, cette propriété est essentielle pour éviter les écarts liés aux différences de versions de Python, de bibliothèques ou de système d'exploitation, et pour simplifier le déploiement continu. Docker facilite ainsi le passage du développement à la production avec un niveau de confiance élevé.

5.1.1 Implémentation dans le projet

5.1.2 Architecture mono-conteneur (API + dashboard + Nginx)

Le choix d'un conteneur unique découle directement des contraintes de Hugging Face Spaces, qui n'autorise ni le déploiement multi-conteneurs (pas de Docker Compose), ni l'exposition de plusieurs ports publics (un seul port, 7860). Or l'application comporte deux services distincts : une API FastAPI (interne en 8000) et un dashboard Streamlit (interne en 8501). Pour respecter ce cadre tout en conservant une architecture claire, **Nginx** (serveur web léger capable de faire du *reverse proxy*) est utilisé comme point d'entrée unique : il écoute sur le port exposé et applique un routage par chemin, en redirigeant `api` vers FastAPI et toutes les autres routes vers Streamlit. Cette couche joue aussi le rôle classique de reverse proxy, en masquant l'organisation interne des services et en

unifiant l'accès côté utilisateur. En parallèle, la cohabitation de plusieurs processus dans un même conteneur impose un superviseur : **Supervisor** (gestionnaire de processus qui lance, maintient et redémarre automatiquement des applications) lance, maintient et redémarre si besoin **uvicorn**, **streamlit** et **nginx**. Cette combinaison garantit une exécution fiable malgré l'absence d'orchestrateur, tout en restant compatible avec le modèle de déploiement imposé par la plateforme.

5.1.3 Élaboration du Dockerfile et construction de l'image

Dockerfile :

Le Dockerfile définit la construction reproductible de l'image Docker. Il utilise l'image légère `python:3.13-slim`, installe les paquets système (`nginx`, `supervisor`) ainsi que les dépendances Python. Le code source est copié, la variable `PYTHONPATH` est positionnée, et les fichiers `nginx.conf` et `supervisord.conf` sont placés à leurs emplacements. Enfin, la commande par défaut exécute `supervisord` pour lancer et surveiller l'API, le dashboard et Nginx.

```
Dockerfile > ...
FROM python:3.13-slim

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PIP_NO_CACHE_DIR=1 \
    PROJECT_ROOT=/app \
    PYTHONPATH=/app

WORKDIR /app

RUN apt-get update && apt-get install -y --no-install-recommends \
    nginx \
    supervisor \
    build-essential \
    curl \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt /app/requirements.txt
RUN python -m pip install --upgrade pip && pip install -r /app/requirements.txt

COPY api /app/api
COPY dashboard /app/dashboard
COPY configs /app/configs
COPY data/processed /app/data/processed
COPY models /app/models

COPY nginx.conf /etc/nginx/nginx.conf
COPY supervisord.conf /etc/supervisor/conf.d/supervisord.conf

EXPOSE 7860

HEALTHCHECK --interval=30s --timeout=5s --retries=6 CMD curl -fss http://127.0.0.1:7860/api/health || exit 1

CMD ["supervisord","-n","-c","/etc/supervisor/conf.d/supervisord.conf"]
```

FIGURE 3.11 – Contenu du Dockerfile utilisé dans le projet

Image Docker :

Une image Docker est un modèle immuable et exécutable qui contient tout ce dont une application a besoin pour fonctionner : système d'exploitation minimal, bibliothèques, code source et variables d'environnement. À partir du `Dockerfile`, on construit une image en exécutant la commande `docker build`. Cette opération lit chaque instruction du `Dockerfile`, exécute les étapes (installation, copie, etc.) et sauvegarde le résultat dans une image nommée, prête à être lancée dans un conteneur. Pour ce projet, l'image a été construite localement en utilisant la commande suivante :

```
PS C:\Users\SIGMA IT\Documents\projet_pfa> docker build -f PFA/Dockerfile -t pfa-space:latest PFA
[+] Building 230.7s (8/16)
=> sha256:3531af2bc2a9c8883754652783cf96207d53189db279c9637b7157d034de7ecd 29.78MB / 29.78MB
=> sha256:3f977313cbf20d1fcbf9d25f28735d97b7bc691319bc43f8c539c8d05c9cf886 1.29MB / 1.29MB
=> sha256:eb7aeb6e31180144e93bad8e32441038f4c4520ec825039827c05fadcc01305f 249B / 249B
=> sha256:a51d847b5d4902dac7726c34078eeb8e64ee8047b67ea02877acabcefi91b07f 11.82MB / 11.82MB
=> extracting sha256:3531af2bc2a9c8883754652783cf96207d53189db279c9637b7157d034de7ecd 0.7s
=> extracting sha256:3f977313cbf20d1fcbf9d25f28735d97b7bc691319bc43f8c539c8d05c9cf886 0.1s
=> extracting sha256:a51d847b5d4902dac7726c34078eeb8e64ee8047b67ea02877acabcefi91b07f 0.3s
=> extracting sha256:eb7aeb6e31180144e93bad8e32441038f4c4520ec825039827c05fadcc01305f 0.0s
=> [internal] load build context 36.2s
=> transferring context: 759.65MB 36.2s
[ 2/12] WORKDIR /app 0.2s
[ 3/12] RUN apt-get update && apt-get install -y --no-install-recommends nginx supervisor build-essential curl & 45.9s
[ 4/12] COPY requirements.txt /app/requirements.txt 0.1s
[ 5/12] RUN python -m pip install --upgrade pip && pip install -r /app/requirements.txt 166.3s
=> # Downloading tonl-0.10.2-py2.py3-none-any.whl (16 kB)
=> # Downloading tornado-6.5.5-cp39-abi3-manylinux1_x86_64.manylinux_2_28_x86_64.manylinux_2_5_x86_64.whl (447 kB)
=> # Downloading typing_extensions-4.15.0-py3-none-any.whl (44 kB)
=> # Downloading urllib3-2.6.3-py3-none-any.whl (131 kB)
=> # Downloading watchdog-6.0.0-py3-none-manylinux2014_x86_64.whl (79 kB)
=> # Downloading torch-2.11.0-cp313-cp313-manylinux_2_28_x86_64.whl (530.7 MB)
```

FIGURE 3.12 – Contenu du Dockerfile utilisé dans le projet

5.1.4 Orchestration interne : Nginx et Supervisor

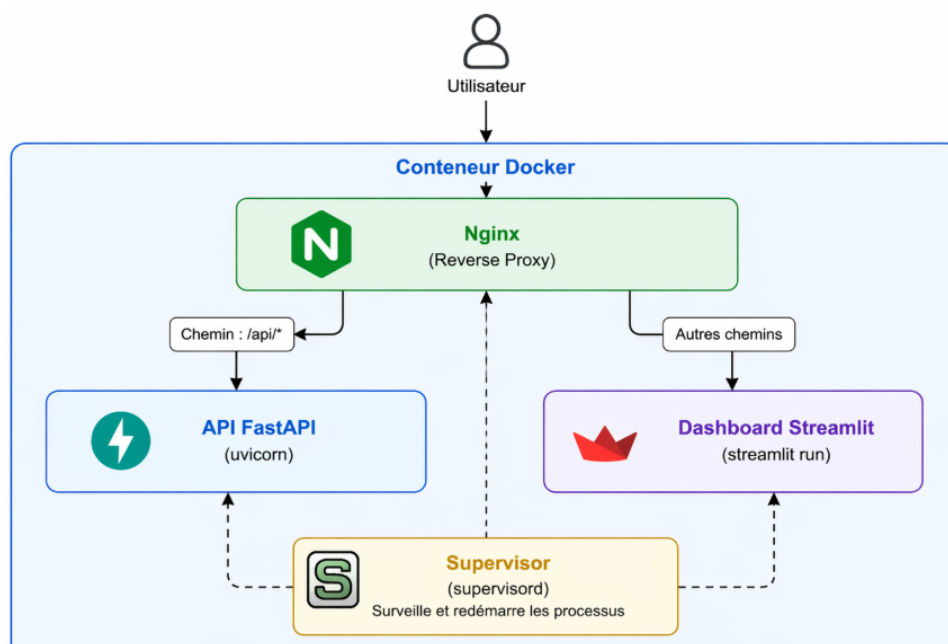


FIGURE 3.13 – Orchestration interne

Nginx :

Nginx agit comme reverse proxy. Il écoute sur le port 7860 et redirige les requêtes `/api` vers l'API (port 8000) et les autres vers Streamlit (port 8501).

Supervisor :

Supervisor est le gestionnaire de processus. Il lance et surveille les trois services (uvicorn, streamlit, nginx) avec redémarrage automatique en cas d'arrêt.

5.1.5 Construction et test local de l'image

```
(venv) PS C:\Users\SIGMA IT\Documents\projet_pfa> docker run --name pfa-space-test --rm -p 7860:7860 pfa-space:latest
2026-04-26 15:47:20,235 CRIT Supervisor is running as root. Privileges were not dropped because no user is specified in the config file. If you intend to run as root, you can set user=root in the config file to avoid this message.
2026-04-26 15:47:20,236 INFO supervisor started with pid 1
2026-04-26 15:47:21,240 INFO spawned: 'api' with pid 7
2026-04-26 15:47:21,243 INFO spawned: 'dashboard' with pid 8
2026-04-26 15:47:21,244 INFO spawned: 'nginx' with pid 9
```

FIGURE 3.14 – Commande de lancement du conteneur en local

L'image Docker a été construite localement à partir du Dockerfile. Un conteneur a ensuite été lancé pour valider le bon fonctionnement de l'API et du dashboard. Les figures ci-dessous présentent respectivement la réponse de l'endpoint de vérification de santé et l'interface du dashboard accessible lors du test local.

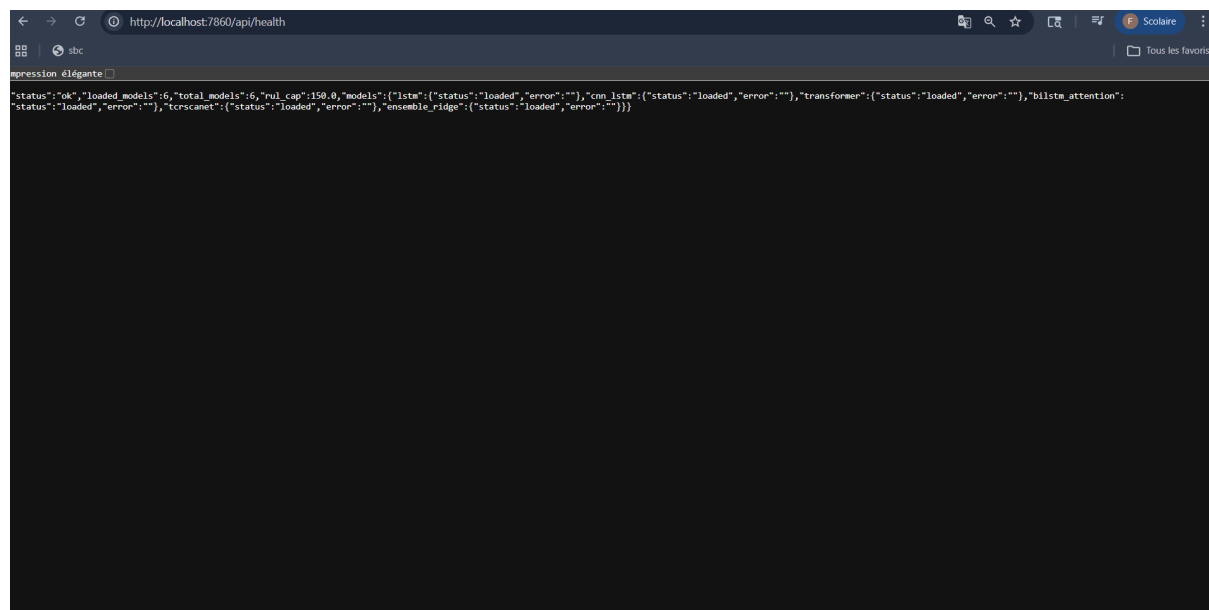


FIGURE 3.15 – Réponse de l'API `/api/health` en local

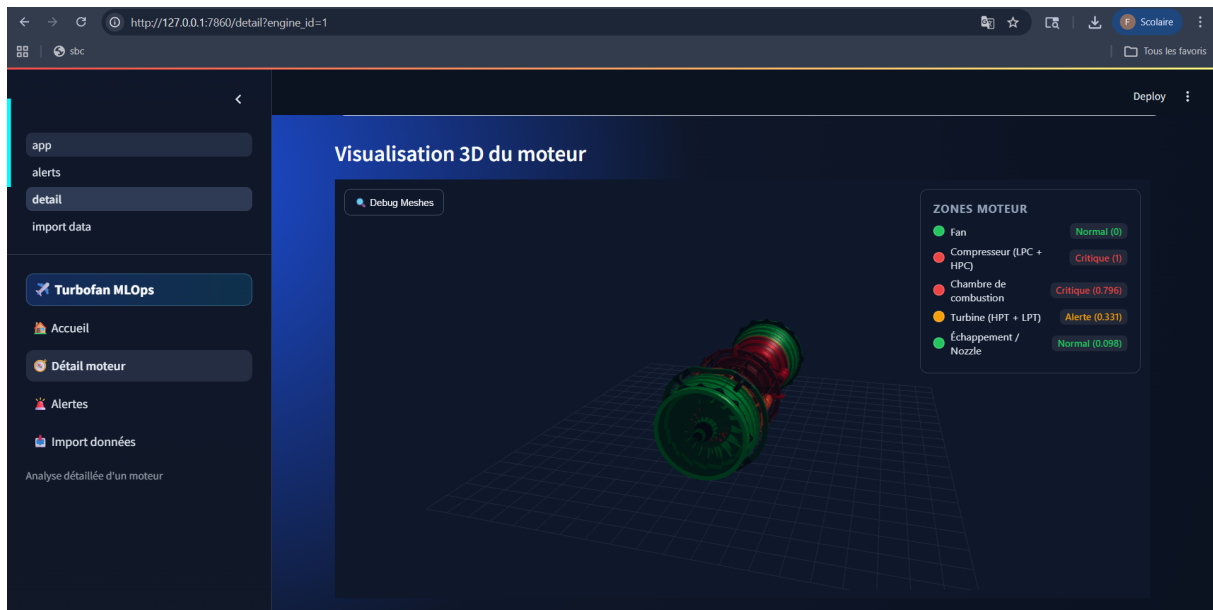


FIGURE 3.16 – Dashboard Streamlit accessible en local

6 Déploiement sur Hugging Face Spaces

6.1 Création du Space Docker et configuration du dépôt

La création d'un Space Docker sur Hugging Face s'effectue via l'interface web en choisissant le type **Docker** lors de la création d'un nouvel espace. La plateforme génère alors automatiquement un dépôt Git distant et un fichier de configuration minimal, prêt à recevoir le code conteneurisé.

The screenshot displays the Hugging Face Spaces creation interface. At the top, the 'Owner' is set to 'feres007' and the 'Space name' is 'New Space name'. Below these are fields for 'Short description' and 'License'. The 'Select the Space SDK' section offers three options: 'Gradio' (4 templates), 'Docker' (17 templates, highlighted with a blue border), and 'Static' (6 templates). Under 'Choose a Docker template:', a grid of 16 templates is shown, including 'Blank', 'Streamlit', 'JupyterLab', 'Argilla', 'Livebook', 'LabelStudio', 'AimStack', 'Shiny (R)', 'Shiny (Python)', 'ZenML', 'ChatUI', 'Panel', 'Giskard', 'Quarto', 'marimo', 'Evidence', 'Langfuse', and 'Plotly'. The 'Space hardware' section, marked 'Free', shows 'CPU Basic' as the selected option, with 'ZeroGPU' and 'Custom Hardware' as alternatives. The 'Storage Bucket' section has a toggle for 'Mount a bucket to this Space'. The 'Space Dev Mode' section, marked 'BETA' and 'PRO subscribers', includes a toggle for 'Enable dev mode' and a link to 'Subscribe to PRO to use dev mode.'. At the bottom, the 'Public' visibility option is selected, with a note: 'Anyone on the internet can see this Space. Only you can commit.'

FIGURE 3.17 – Création du Space Docker

6.2 Versionnement et push du code via Git

6.2.1 Initialisation du dépôt local et liaison avec le Space

Le projet est d'abord versionné localement à l'aide de la commande `git init`. Le dépôt distant du Space Hugging Face est ensuite ajouté comme remote via `git remote add space <URL>`. La commande `git remote -v` permet de vérifier que la liaison est correctement établie.

```
(venv) PS C:\Users\SIGMA IT\Documents\projet_pfa> git init
Reinitialized existing Git repository in C:/Users/SIGMA IT/Documents/projet_pfa/.git/
(venv) PS C:\Users\SIGMA IT\Documents\projet_pfa>
```

FIGURE 3.18 – Initialiser un dépôt Git local

```
(venv) PS C:\Users\SIGMA IT\Documents\projet_pfa> git add .
(venv) PS C:\Users\SIGMA IT\Documents\projet_pfa> git commit -m "Initial commit"
[main 2333926] Initial commit
1 file changed, 1 insertion(+)
create mode 160000 PFA
```

FIGURE 3.19 – Ajouter tous les fichiers

```
(venv) PS C:\Users\SIGMA IT\Documents\projet_pfa> git remote add space https://huggingface.co/spaces/feres007/PFA
(venv) PS C:\Users\SIGMA IT\Documents\projet_pfa> git remote -v
origin https://github.com/Feres0007/TurboFan_Project_PFA.git (fetch)
origin https://github.com/Feres0007/TurboFan_Project_PFA.git (push)
space https://huggingface.co/spaces/Feres007/PFA (fetch)
space https://huggingface.co/spaces/feres007/PFA (push)
```

FIGURE 3.20 – Ajouter le dépôt distant du Space Hugging Face

6.2.2 Push du code et déclenchement automatique du build

Une fois le remote configuré, l'intégralité du code source est envoyée vers le dépôt distant par `git push space main`. Hugging Face détecte alors automatiquement le `Dockerfile` et lance la construction de l'image. Les logs de ce build sont consultables en temps réel dans l'onglet `Settings` du Space, sous la section `Container` ou `Build logs`. À l'issue du build réussi, l'application est accessible à l'URL publique du Space.

```
(venv) PS C:\Users\SIGMA IT\Documents\projet_pfa> git push space main
```

FIGURE 3.21 – Pousser le code vers la branche principale du Space

6.2.3 Suivi du build et analyse des logs

Dès que le code est poussé sur le dépôt distant, Hugging Face déclenche automatiquement la construction de l'image Docker. Cette opération est entièrement transparente pour l'utilisateur : l'intégralité des étapes (installation des dépendances, copie des fichiers, exécution des commandes) est enregistrée dans des logs accessibles depuis l'onglet **Settings** du Space, sous la section **Container** ou **Build logs**. La consultation de ces logs permet de suivre l'avancement du build en temps réel et de s'assurer que chaque instruction du **Dockerfile** s'exécute correctement. En cas d'anomalie, les erreurs sont clairement affichées, ce qui facilite leur identification et leur correction. Un build réussi se termine par un message de validation, après quoi l'application devient accessible à l'URL publique du Space.

```
==== Build Queued at 2026-04-20 18:17:08 / Commit SHA: a498b4f =====
--> FROM docker.io/library/python:3.13-slim@sha256:d168b8d9eb761f4d3fe305ebd04aeb7e7f2de0297cec5fb2f8f6403244621664
DONE 0.0s

--> COPY dashboard /app/dashboard
CACHED

--> RUN apt-get update && apt-get install -y --no-install-recommends nginx supervisor build-essential curl && rm -rf /var/lib/apt/lists/*
CACHED

--> RUN python -m pip install --upgrade pip && pip install -r /app/requirements.txt
CACHED

--> COPY api /app/api
CACHED

--> COPY configs /app/configs
CACHED

--> WORKDIR /app
CACHED

--> COPY requirements.txt /app/requirements.txt
CACHED

--> COPY data/processed /app/data/processed
CACHED

--> COPY models /app/models
CACHED

--> COPY nginx.conf /etc/nginx/nginx.conf
CACHED

--> Restoring cache
DONE 24.9s

--> COPY supervisord.conf /etc/supervisor/conf.d/supervisord.conf
DONE 0.0s

--> Pushing image
DONE 0.6s

--> Exporting cache
DONE 0.2s
```

FIGURE 3.22 – logs

7 Validation des modèles et analyse des résultats

7.1 Résultats comparatifs

L'évaluation quantitative porte sur les quatre blocs fonctionnels suivants : prédiction du RUL, classification des conditions opérationnelles, détection d'anomalies et explicabilité

par modèle substitut. Les modules de drift et de recommandations sont exclus du périmètre de cette analyse. Pour la régression (RUL), les métriques retenues sont le RMSE, le MAE et le NASA Score (plus les valeurs sont faibles, meilleure est la performance). La classification des conditions opérationnelles est mesurée par l'accuracy et le F1-score. La détection d'anomalies utilise le F1-score et la ROC-AUC. Enfin, la fidélité du modèle substitut (XAI) est appréciée par le RMSE.

7.1.1 Tableau des performances

TABLE 3.2 – Performances des modèles sur le test externe (hors drift et recommandations)

Bloc	Modèle	RMSE	MAE	Accuracy	F1
RUL	Transformer	20,284	14,048	—	—
RUL	hybride	21,129	15,557	—	—
RUL	BiLSTM-Attention	21,491	15,798	—	—
RUL	CNN-LSTM	22,610	16,292	—	—
RUL	TCRSCA-Net	24,012	17,500	—	—
RUL	LSTM	28,174	21,544	—	—
Conditions	Random Forest	—	—	1,000	1,000
Anomalies	Isolation Forest	—	—	—	0,121
XAI (surrogate)	Random Forest Regressor	20,224	—	—	—

7.2 Analyse approfondie des résultats par modèle

7.2.1 Modèle LSTM

Le LSTM simple obtient les performances les plus faibles (RMSE 28,17, MAE 21,54). Cette architecture peinerait à capturer les dépendances temporelles longues dans les séquences de 50 cycles. Les erreurs sont particulièrement élevées pour les fenêtres où le RUL est inférieur à 30 cycles, indiquant une difficulté à anticiper la fin de vie imminente. Le NASA Score très élevé (13464,7) confirme une sensibilité excessive aux surestimations tardives.

7.2.2 Modèle CNN-LSTM

Le CNN-LSTM améliore nettement le LSTM simple (RMSE 22,61, MAE 16,29). Les couches convolutives permettent d'extraire des motifs locaux dans les séries temporelles, ce qui aide à détecter les premiers signes de dégradation. Cependant, le modèle reste moins performant que le Transformer, notamment sur les séquences où les capteurs présentent des variations brutales.

7.2.3 Modèle Transformer

Le Transformer atteint les meilleures métriques (RMSE 20,28, MAE 14,04). Le mécanisme d'attention multi-têtes permet de pondérer différemment chaque cycle et chaque capteur, capturant ainsi des relations à long terme que les LSTM ne peuvent pas modéliser efficacement. Le pourcentage de prédictions à moins de 10 cycles d'erreur est de 55,7%, ce qui est satisfaisant pour une tâche de pronostic sur données synthétiques. Le modèle est donc retenu pour l'API.

7.2.4 Analyse des erreurs résiduelles

Une analyse des résidus (écart entre prédiction et RUL réel) montre que les erreurs sont globalement centrées autour de zéro, mais avec une légère tendance à la sous-estimation pour les moteurs en fin de vie ($RUL < 20$ cycles). Ce biais est acceptable dans un contexte de maintenance prédictive car une sous-estimation conduit à une alarme un peu trop précoce, ce qui est moins grave qu'une surestimation (qui pourrait entraîner une panne non anticipée).

8 Conclusion

Ce chapitre final a démontré comment déployer une solution de Machine Learning en appliquant les principes de MLOps. L'architecture modernisée — combinant API REST, dashboard interactif, et conteneurisation Docker — assure une expérience utilisateur de qualité professionnelle tout en maintenant une séparation claire entre les services.

Le déploiement sur Hugging Face Spaces a permis de rendre la solution accessible sans infrastructure locale complexe. Les tests locaux ont confirmé que les endpoints de l'API répondent rapidement et que le dashboard Streamlit offre une interface intuitive pour visualiser les prédictions et les métriques.

Conclusion Générale

Ce travail avait pour objectif la conception et l'implémentation d'un pipeline MLOps complet dédié à la maintenance prédictive d'un système mécanique, en particulier un turboréacteur. Une étude approfondie des architectures de Deep Learning et des principes de l'ingénierie MLOps a permis de poser les bases nécessaires pour répondre aux enjeux de sûreté et de disponibilité dans le secteur aéronautique.

Dans ce cadre, une étude comparative a été menée sur sept architectures de réseaux de neurones profonds (LSTM, CNN-1D, BiLSTM, Transformer et modèles hybrides) en utilisant le jeu de données C-MAPSS de la NASA. Les résultats obtenus montrent la supériorité du modèle BiLSTM avec un RMSE de 21.49, suivi du modèle hybride CNN-LSTM-Transformer avec un RMSE de 21.129 .

Au-delà des performances des modèles, ce travail a permis l'industrialisation complète de la solution à travers un pipeline MLOps incluant l'ingestion des données, la conteneurisation avec Docker, le déploiement d'une API REST via FastAPI, ainsi qu'un dashboard interactif développé avec Streamlit. L'intégration de mécanismes de monitoring et de détection de dérive renforce la robustesse et la reproductibilité du système en environnement de production.

Malgré les défis rencontrés, notamment lors du prétraitement des données et de la gestion des fuites d'information, ce projet nous a permis d'explorer les enjeux concrets du Deep Learning appliqué à l'industrie et de l'ingénierie MLOps. L'approche modulaire adoptée constitue ainsi une base solide pour une maintenance intelligente et proactive.

En perspectives, ce travail pourrait être enrichi par l'intégration de techniques d'AutoML pour l'optimisation des hyperparamètres, l'extension à des systèmes multi-machines, ainsi que par l'utilisation plus poussée de l'intelligence artificielle explicable (XAI).

Bibliographie

- [1] LYES, Atmimou et HASSINA, Belfaked. Système automatique de production et adaptation d'un automate à une machine de l'électro-industrie. 2011. Thèse de doctorat. Université Mouloud Mammeri.
- [2] CHARTRAND, Alexandre. Détection des défauts dans un environnement de maintenance prédictive. 2021. Thèse de doctorat. École de technologie supérieure.
- [3] HADIL, MAACHI et ANIS, ZOURAGH. Prédiction des pannes d'un système mécanique par apprentissage automatique. 2024. Thèse de doctorat. université Ibn Khaldoun.
- [4] LOPEZ, Myriam. Analyse et prédiction d'erreurs critiques dans une application industrielle. 2023. Thèse de doctorat. Université de Bordeaux.
- [5] LAHOUEL, Ali TAHA EL et LARBAOUI, AbdelBasset. Utilisation des méthodes de l'intelligence artificielle pour la maintenance prédictive : application à l'industrie automobile. 2025. Thèse de doctorat. Directeur : Mr. Fouad MALIKI.
- [6] GUILLAUME, Antoine, VRAIN, Christel, et ELLOUMI, Wael. Vers un outil d'évaluation comparative pour la maintenance prédictive : comment comparer différentes approches?. In : EGC 2023-Atelier GATS. 2023.
- [7] GAURIAT, Charles-Maxime. Vers une maintenance prévisionnelle interprétable par modèles d'apprentissage automatique dans les systèmes complexes à composants tournants. 2025. Thèse de doctorat. Université de Toulouse.
- [8] DEHGHAN SHOORKAND, Hassan. Deep learning for dynamic integration of data-driven predictive maintenance and production planning. 2024.
- [9] YAHIAOUI, Lydia, MOHAMMEDI, Mohamed, et HAMOUID, Khaled. Efficient and Reliable Predictive Maintenance in Trains based on BiLSTM Model. In : 2025 Inter-

- national Wireless Communications and Mobile Computing (IWCMC). IEEE, 2025. p. 149-154.
- [10] SAXENA, Abhinav, GOEBEL, Kai, SIMON, Don, et al. Damage propagation modeling for aircraft engine run-to-failure simulation. In : 2008 international conference on prognostics and health management. IEEE, 2008. p. 1-9.
- [11] KREUZBERGER, Dominik, KÜHL, Niklas, et HIRSCHL, Sebastian. Machine learning operations (mlops) : Overview, definition, and architecture. IEEE access, 2023, vol. 11, p. 31866-31879.
- [12] Carroll, J., Koukoura, S., McDonald, A., Charalambous, A., Weiss, S., McArthur, S. (2018). "Wind turbine gearbox failure and remaining useful life prediction using machine learning techniques." Wind Energy, Vol. 22(3), pp. 360-375.
- [13] ASMAA, MOTRANI. Contribution à la mise en œuvre du Prognostic and Health Management (PHM) industriel. 2022. Thèse de doctorat. Université d'Oran.

